



x_{1_1}



x_{1_2}



x_{2_1}



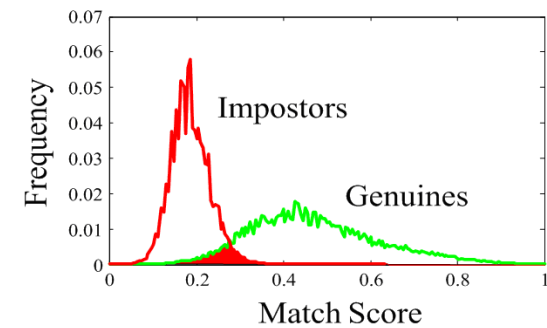
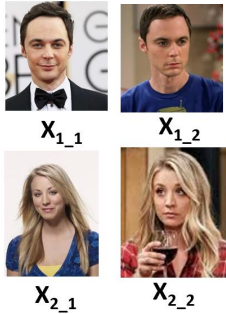
x_{2_2}

Tecniche e applicazioni biometriche

Fabio Scotti

fabio.scotti@unimi.it

Università degli Studi di Milano



Parte A

Lezione

Modelli e metodi statistici per la biometria

Fabio Scotti
fabio.scotti@unimi.it

Università degli Studi di Milano

L9 A



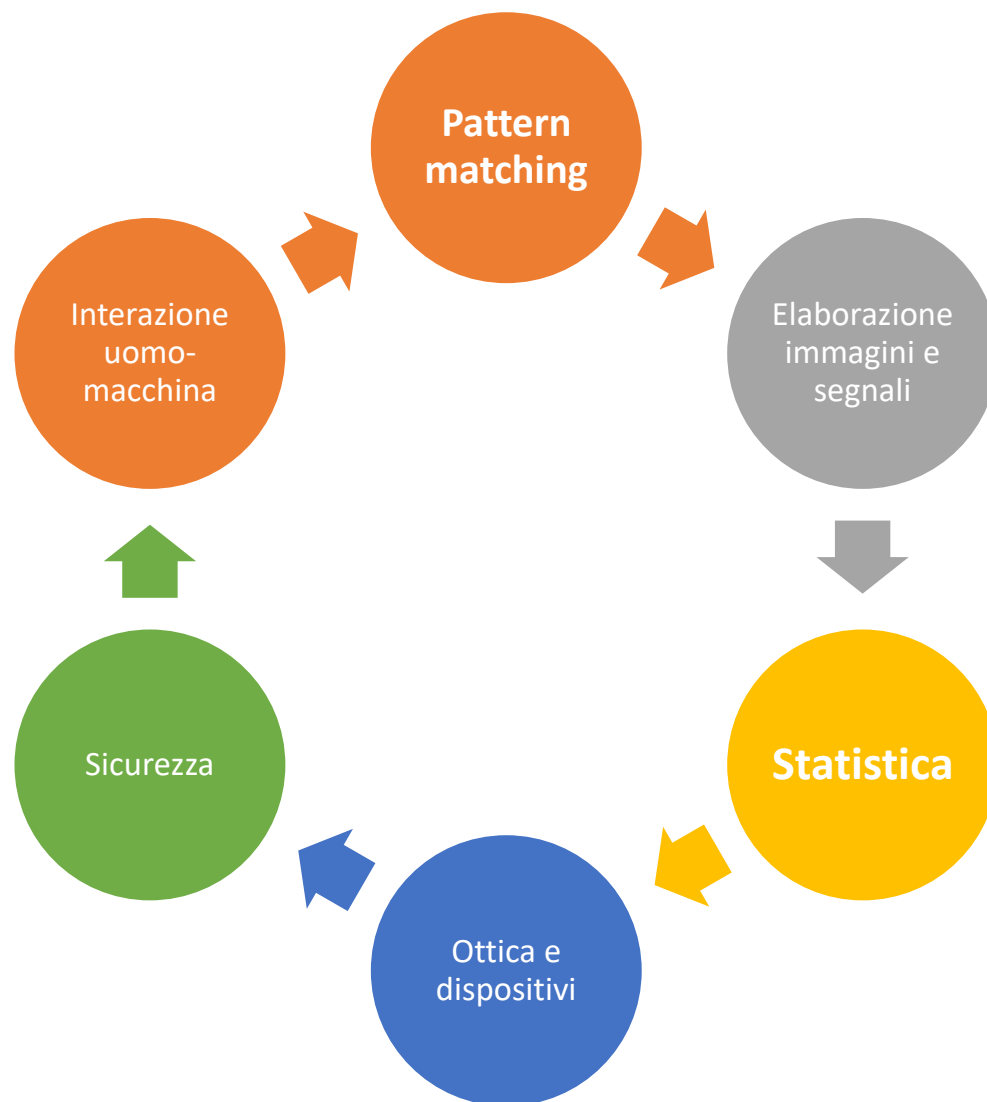
Outline

DATA REPORTING sistemi biometrici

1. Misura della accuratezza dei sistemi biometrici
2. Modelli e metodi statistici applicabili

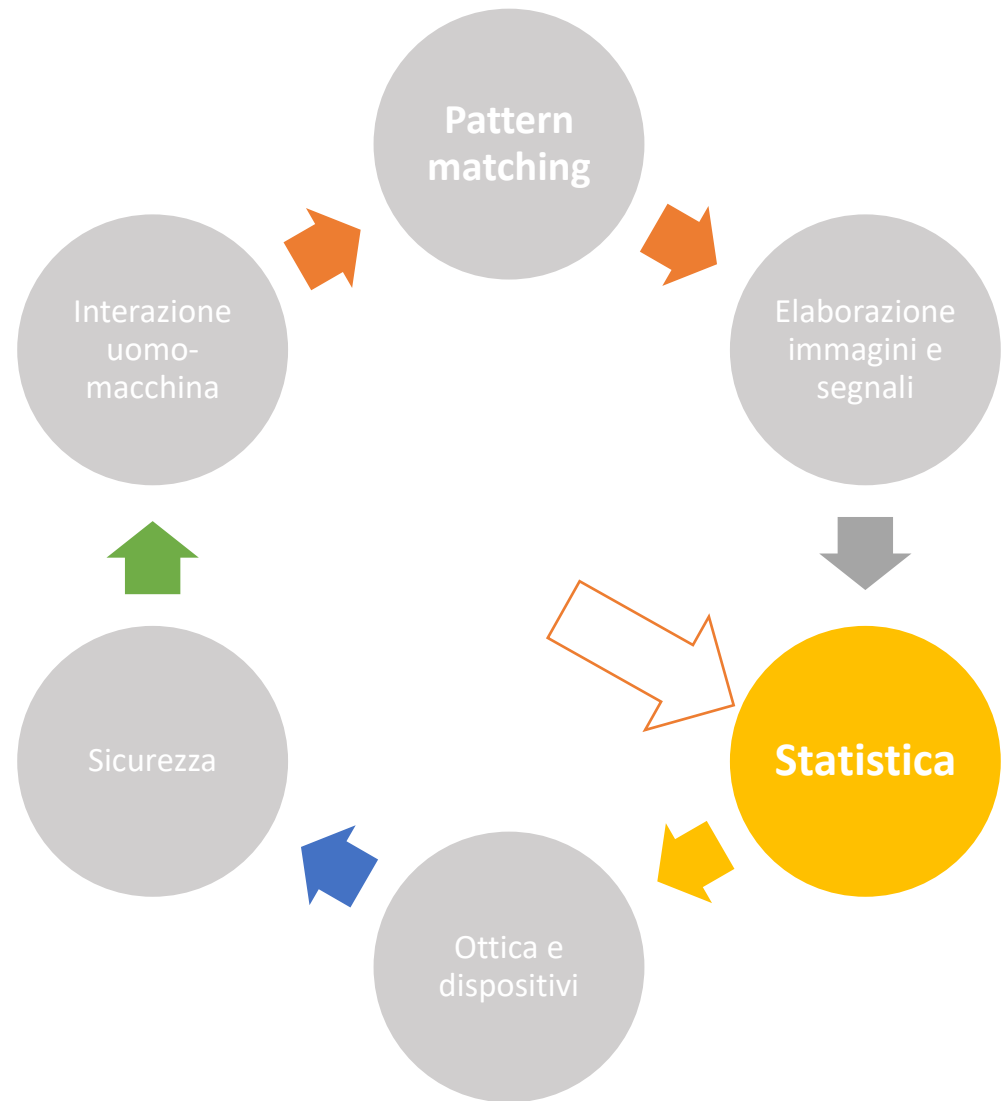
Conoscenza
nei sistemi
biometrici

Aspetti
multi-
disciplinari



Conoscenza
nei sistemi
biometrici

Aspetti
multi-
disciplinari



Esamineremo quali tecniche usare per
capire come si **comporta** un sistema biometrico,
come **regolarlo** e come eseguire il **reporting**

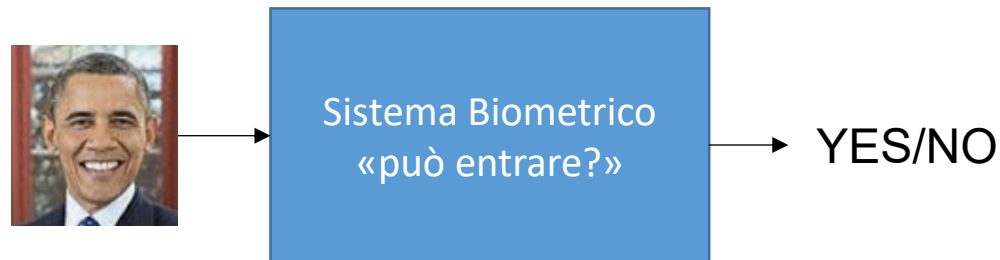


Modelli e metodi statistici per la misura dei sistemi biometrici

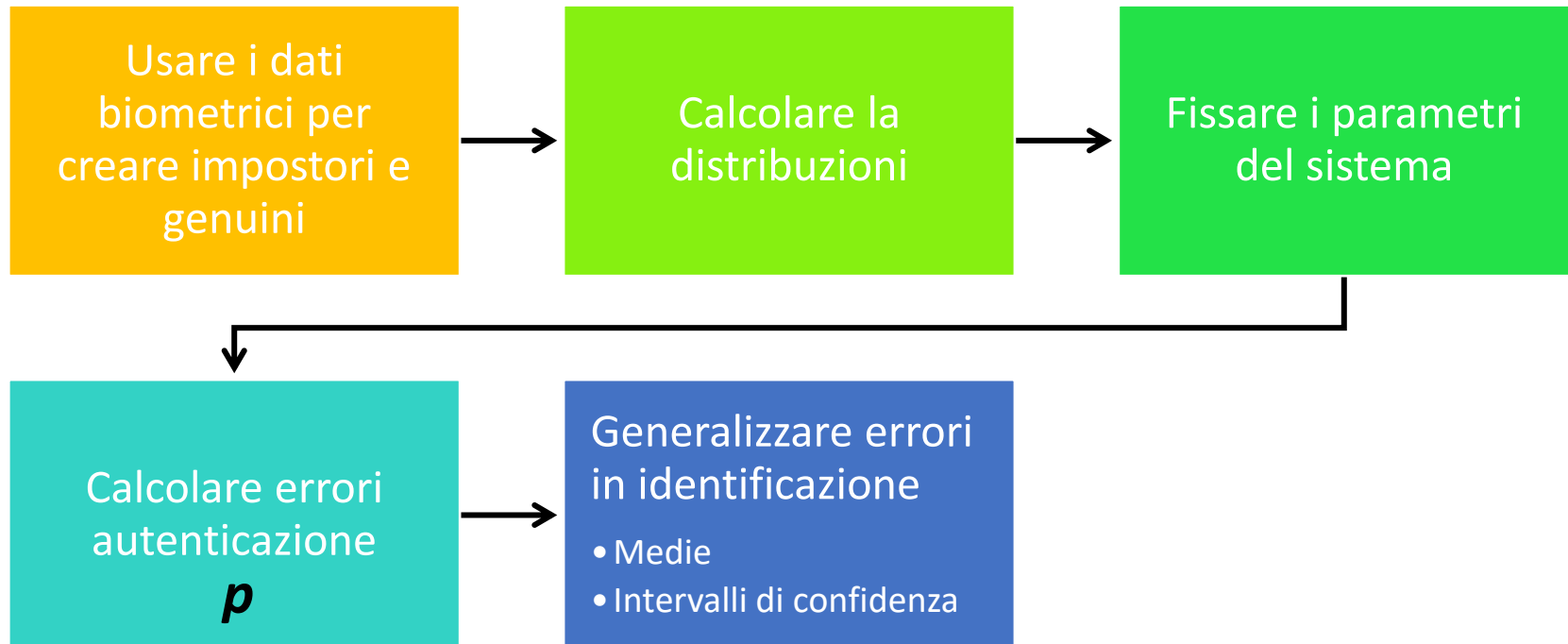
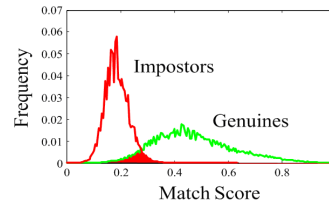
Metriche, Uso dei dati, Reporting

Premessa: sistema configurato

- Se il sistema ha una soglia T variabile → dobbiamo analizzare la curva DET e regolare la sua soglia in base alla applicazione.
- Il sistema completamente fissato si comporta come un **algoritmo dipendente dai dati** (input).
- A questo punto possiamo fare la valutazioni sulla sua accuratezza e comportamento statistico



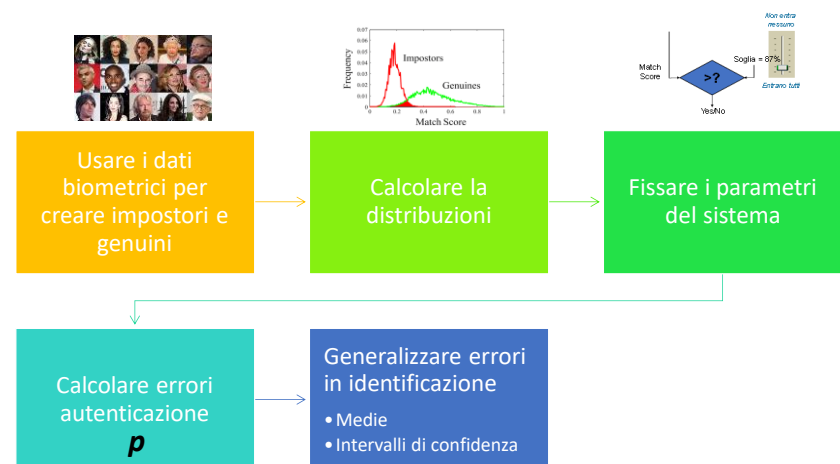
Flusso di lavoro



Comprare un sistema «of the shelf»

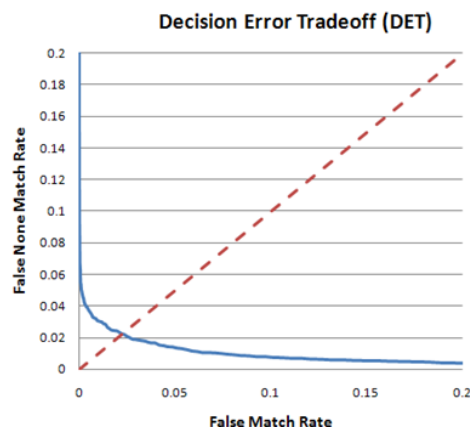
Si possono avere diversi casi:

- Avere già p dal produttore
- Scegliere solo T , altri parametri?
- Fidarsi completamente dei dati riportati sui datasheet (?!)
- Oppure fare esperimenti?



EER o p(errore) ?

Meglio usare i grafici della DET senza ragionare sulla soglia oppure configurare tutto il sistema e studiare il suo errore (la prob. di errore)?



Sistema Biometrico
«può entrare?»
Parametri fissati

YES/NO

Meglio integrare tutte le informazioni in un documento ragionato

→→ supporto per decisioni future o modifiche

Uso i miei dati o dei modelli statistici?

L'inferenza statistica (statistica)

Procedimento per cui si inducono le caratteristiche di una popolazione dall'osservazione di una parte di essa (detta "campione") mediante un esperimento casuale (aleatorio)

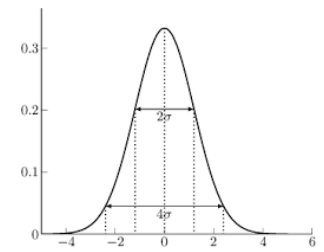
Calcolo delle probabilità

E' noto il processo di generazione dei dati sperimentali (modello probabilistico)

→ probabilità dei diversi possibili risultati di un esperimento

«Volontari, Conteggi, Sommatorie, Foglio Excel»

«Datasheet, Libri, Integrali, R, Matlab»



Non ho nessun dato → Uso volontari per le stimare le distribuzioni dei match score

- Sia $X_{i,j}$ il j -esimo template dell'individuo i -esimo



$X_{1,1}$



$X_{1,2}$

Match Genuini

$$S(X_{1,1}, X_{1,2}) = 0.7$$

$$S(X_{1,1}, X_{1,3}) = 0.8$$

$$S(X_{2,1}, X_{2,2}) = 0.4$$

$$S(X_{2,1}, X_{2,3}) = 0.5$$

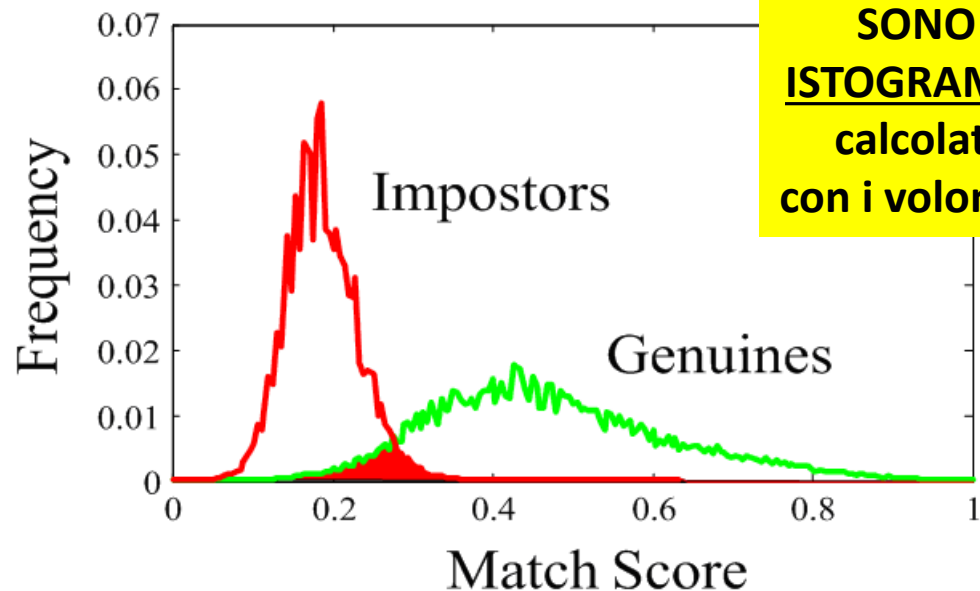
Match Impostori

$$S(X_{1,1}, X_{3,2}) = 0.11$$

$$S(X_{4,1}, X_{3,1}) = 0.21$$

$$S(X_{5,2}, X_{1,2}) = 0.001$$

$$S(X_{2,2}, X_{1,2}) = 0.19$$

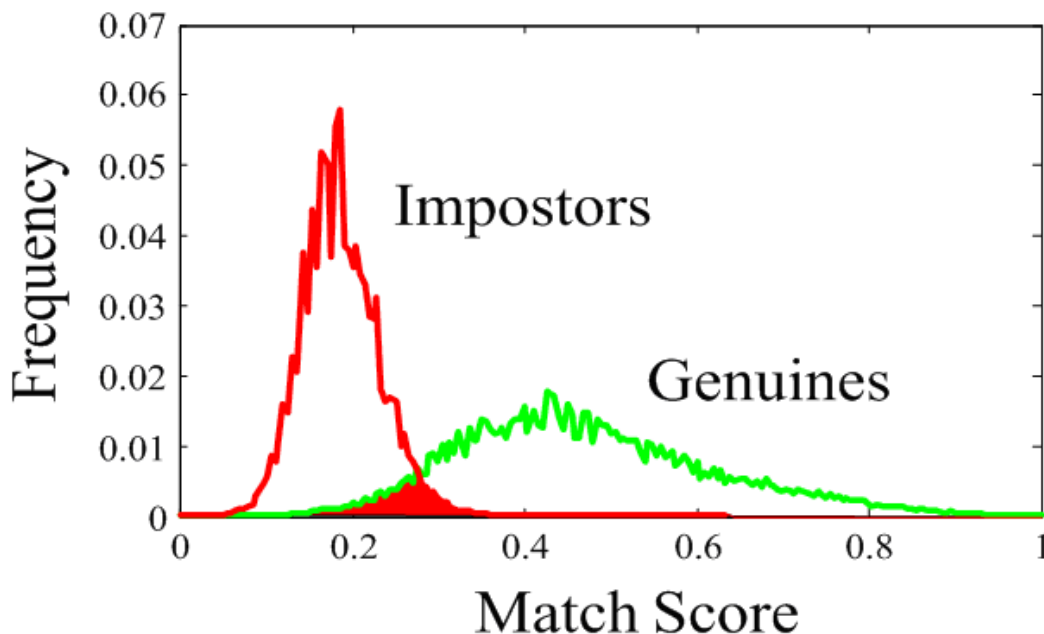


**SONO
ISTOGRAMMI
calcolati
con i volontari**

DATA REPORTING

LE DISTRIBUZIONI matchscore_{/distanze}

- Ecco il primo e fondamentale dato da ricavare!
 - Controllare con esperimenti se disponibile dai datasheet....



Cerco sempre di ottenerlo da....

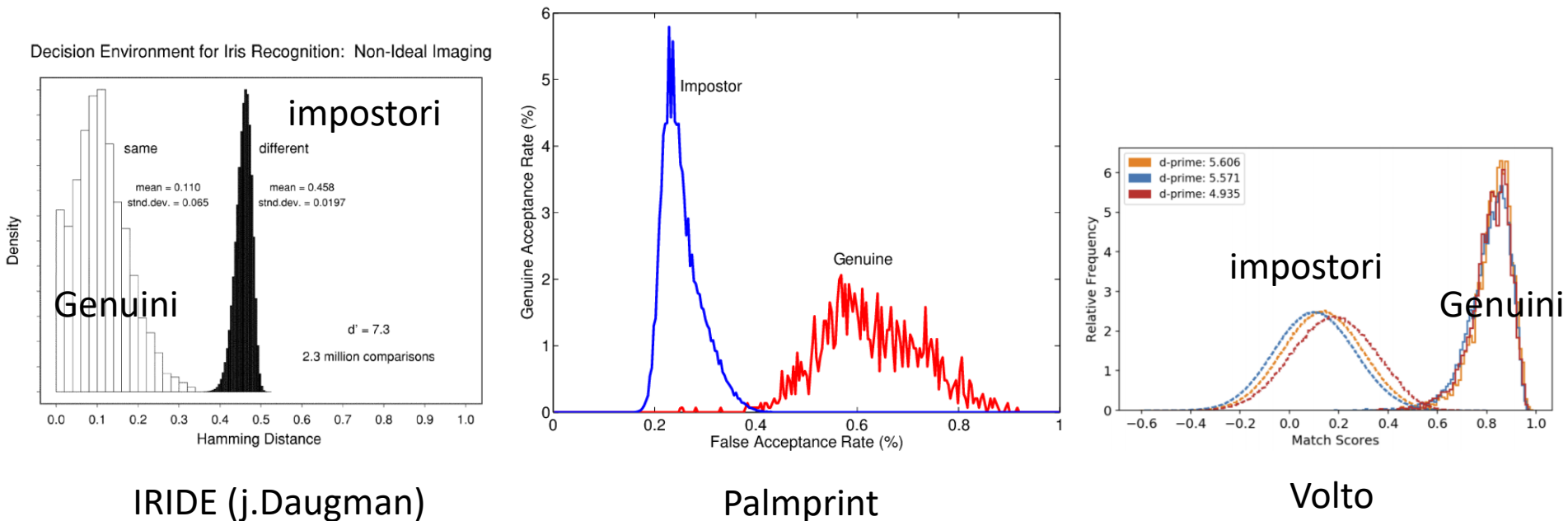
- interpolazione dal datasheet
- dalla rivista scientifica
- dai miei dati personali precedenti
- da una valutazione
 - Technology,
 - Scenario,
 - Operational
- dagli esperimenti con i volontari

Caratteristiche generali delle distribuzioni genuini ed impostori

(nel caso da inferenza statistica = dati veri)

DOMANDA: perché i genuini sono più «rumorosi» degli impostori (o rappresentati a barre)??

RIS.: perché **la cardinalità è assolutamente diversa!**



Cardinalità genuini

(nel caso da inferenza statistica = dati veri)

2 immagini/utente

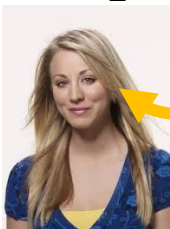


X_{1_1}



X_{1_2}

1 match



X_{2_1}



X_{2_2}

1 match

...

N utenti

Supponendo **2 foto per individuo**
e in generale un matching simmetrico
 $S(X_{1_1}, X_{1_2}) = S(X_{1_2}, X_{1_1})$

Match Genuini

$$S(X_{1_1}, X_{1_2}) = 0.7$$

$$S(X_{2_1}, X_{2_2}) = 0.8$$

....

$$S(X_{N_1}, X_{N_2}) = 0.5$$

match score genuini = N
(2 foto per ind.)

Cardinalità **impostori**

(nel caso da inferenza statistica = dati veri)

2 immagini/utente

Matching simmetrico

Match Impostori X_{1_1}

$$S(X_{1_1}, X_{2_1}) = 0.11$$

$$S(X_{1_1}, X_{2_2}) = 0.12$$

$$S(X_{1_1}, X_{3_1}) = 0.21$$

$$S(X_{1_1}, X_{3_2}) = 0.22$$

....

$$S(X_{1_1}, X_{N_1}) = 0.44$$

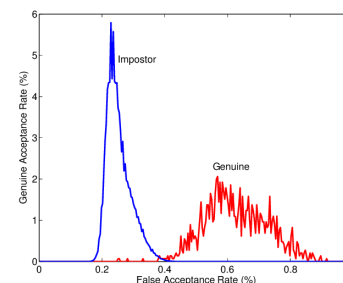
$$S(X_{1_1}, X_{N_2}) = 0.43$$

$$\#match = 2(N-1)$$

Iniziamo dalla prima foto del primo individuo
con le foto di tutti gli altri individui



N utenti



Cardinalità **impostori**

(nel caso da inferenza statistica = dati veri)

2 immagini/utente

Matching simmetrico



X_{1_1} X_{1_2}

X_{2_1} X_{2_2}

...

N utenti

Match Impostori X_{1_1}

$$S(X_{1_1}, X_{2_1}) = 0.11$$

$$S(X_{1_1}, X_{2_2}) = 0.12$$

$$S(X_{1_1}, X_{3_1}) = 0.21$$

$$S(X_{1_1}, X_{3_2}) = 0.22$$

....

$$S(X_{1_1}, X_{N_1}) = 0.44$$

$$S(X_{1_1}, X_{N_2}) = 0.43$$

$2(N-1)$

Match Impostori X_{1_2}

$$S(X_{1_2}, X_{2_1}) = 0.11$$

$$S(X_{1_2}, X_{2_2}) = 0.12$$

$$S(X_{1_2}, X_{3_1}) = 0.21$$

$$S(X_{1_2}, X_{3_2}) = 0.22$$

....

$$S(X_{1_2}, X_{N_1}) = 0.44$$

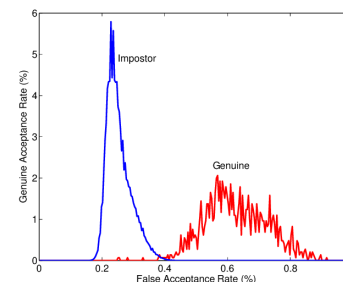
$$S(X_{1_2}, X_{N_2}) = 0.43$$

$2(N-1)$

$4(N-1)$

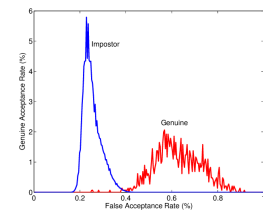
Confronti di impostori
per singolo individuo

Seconda foto del
primo individuo con
le foto di tutti gli altri
individui



Cardinalità **impostori**

(nel caso da inferenza statistica = dati veri)



2 immagini/utente

Matching simmetrico

per tutte le altre foto

Match Impostori X_{1_1}

$$S(X_{1_1}, X_{2_1}) = 0.11$$

$$S(X_{1_1}, X_{2_2}) = 0.12$$

$$S(X_{1_1}, X_{3_1}) = 0.21$$

$$S(X_{1_1}, X_{3_2}) = 0.22$$

....

$$S(X_{1_1}, X_{N_1}) = 0.44$$

$$S(X_{1_1}, X_{N_2}) = 0.43$$

Match Impostori X_{1_2}

$$S(X_{1_2}, X_{2_1}) = 0.11$$

$$S(X_{1_2}, X_{2_2}) = 0.12$$

$$S(X_{1_2}, X_{3_1}) = 0.21$$

$$S(X_{1_2}, X_{3_2}) = 0.22$$

....

$$S(X_{1_2}, X_{N_1}) = 0.44$$

$$S(X_{1_2}, X_{N_2}) = 0.43$$

Match Impostori X_{2_1}

$$S(X_{1_2}, X_{2_1}) = 0.11$$

$$S(X_{1_2}, X_{2_2}) = 0.12$$

$$S(X_{1_2}, X_{3_1}) = 0.21$$

$$S(X_{1_2}, X_{3_2}) = 0.22$$

....

$$S(X_{1_2}, X_{N_1}) = 0.44$$

$$S(X_{1_2}, X_{N_2}) = 0.43$$

X_{2_2}

...

...

...

...

...

...

...

X_{N_2}

...

...

...

...

...

...

...

$4(N-1)$

$4(N-1)$

$4(N-1)$

$N * 4(N-1)$

=

$$4N^2 - 4N = 4(N^2 - N)$$

(due immagini per utente)

RIASSUMENDO!

La cardinalità
degli impostori

$$\approx N^2 - N$$

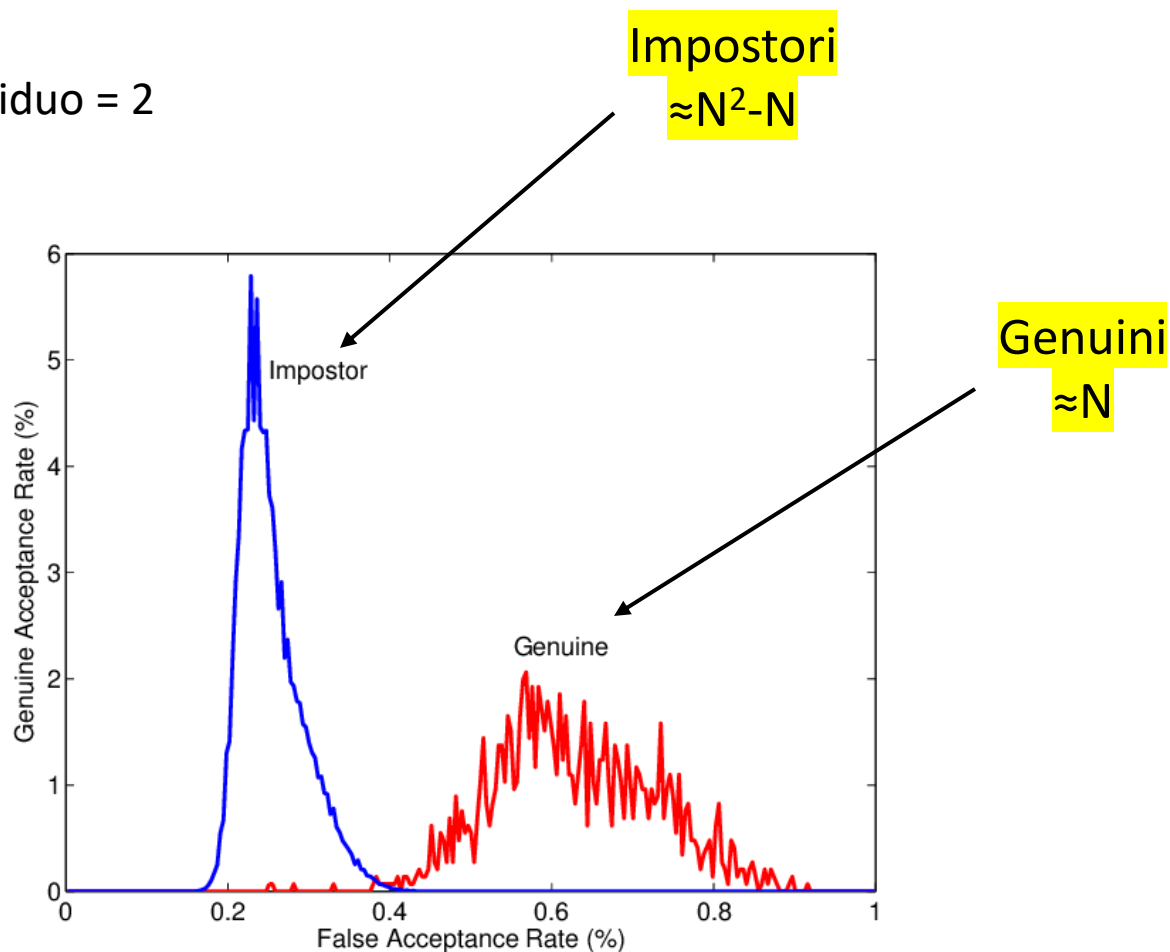
N utenti

Riassunto delle cardinalità

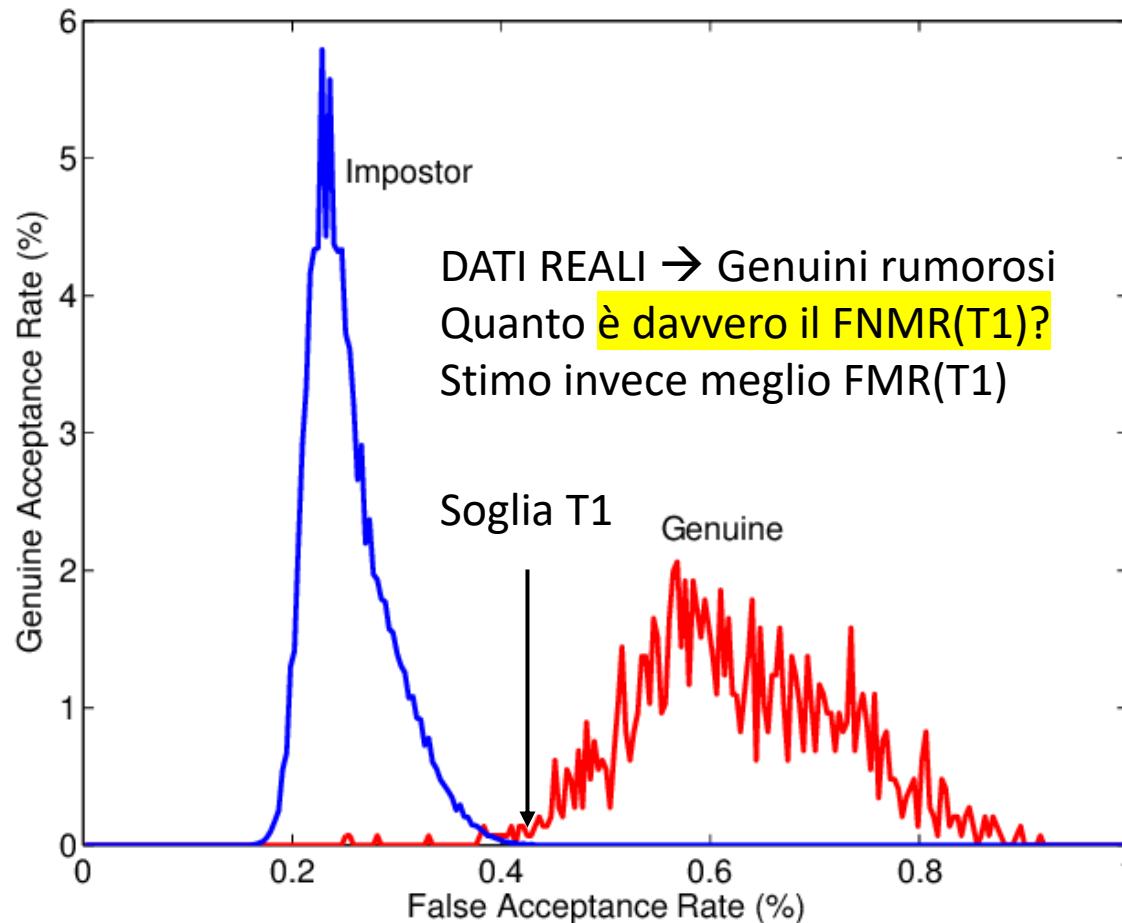
Caso analizzato:

M foto per individuo = 2

N individui

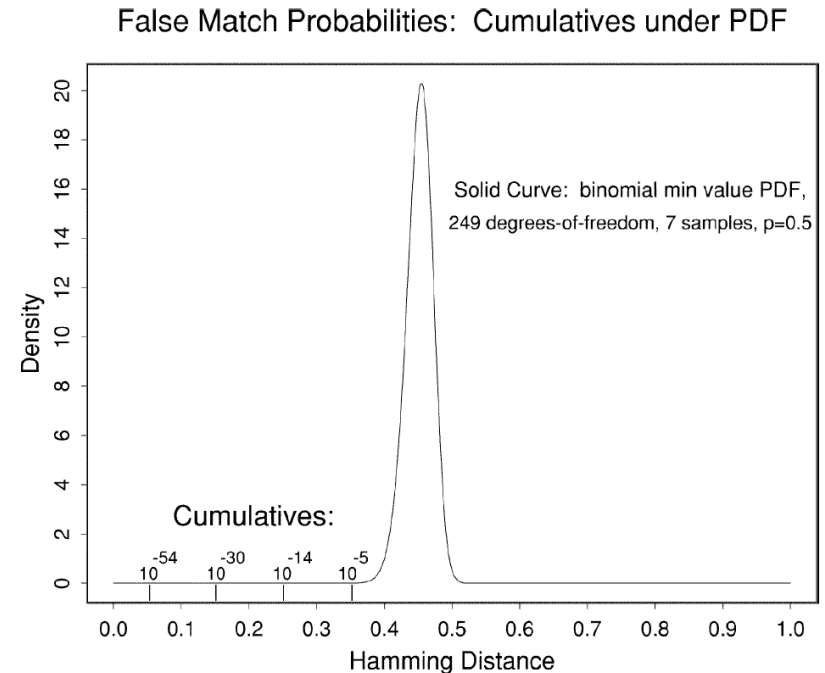


Attenzione alle code delle distribuzioni rumorose calcolando la soglia T



Modello probabilistico/teorico

- Un modello teorico permette di calcolare FMR() e FNMR() in modo preciso
- Ma quanto la mia applicazione è vicina a modello teorico che ha generato le curve delle distribuzioni?
- Ho sia genuini sia imp.?
Non sempre... in caso uso ragionando tutti i dati che riesco a ricavare



Esempio di modello teorico della distribuzione degli impostori per l'Iriscode.

Si possono calcolare probabilità di 10^{-30} impossibili da verificare con esperimenti (10.000.000.....000 persone da trovare)

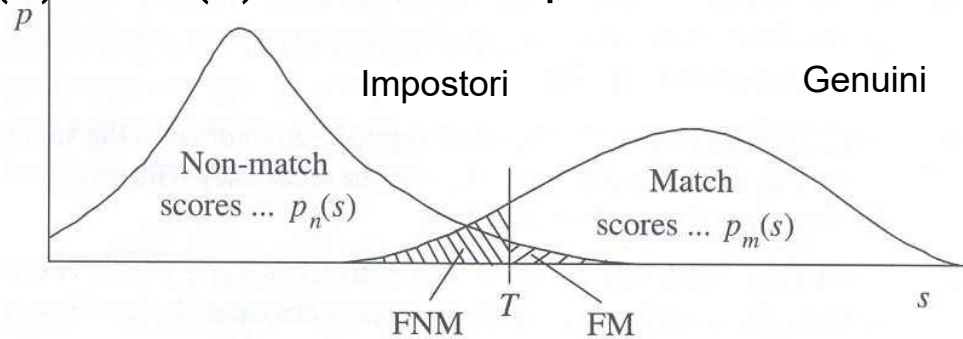
Calcolo diretto dalle distribuzioni → FMR FNMR

- Caso diverso: ho a disposizione le distribuzioni di genuini ed impostori (o le approssimo con gli istogrammi visti prima).
- Supponiamo di poter variare la soglia s e di fissarla ad un valore T in mezzo fra il picco degli impostori e quello dei genuini
- Notiamo che un certo numero di persone appartenenti al gruppo dei genuini sono sotto la soglia T e quindi non saranno autorizzati dando luogo ad errori di False Non-Match (FNM).

$$\text{FNMR}(T) = \text{FNM}(T) / \text{Totale_Genuini}$$

- Al contrario una parte degli impostori hanno valori di match sopra la soglia T e quindi saranno autorizzati dando luogo ad errori di False Match.

$$\text{FMR}(T) = \text{FM}(T) / \text{Totale_Impostori}$$



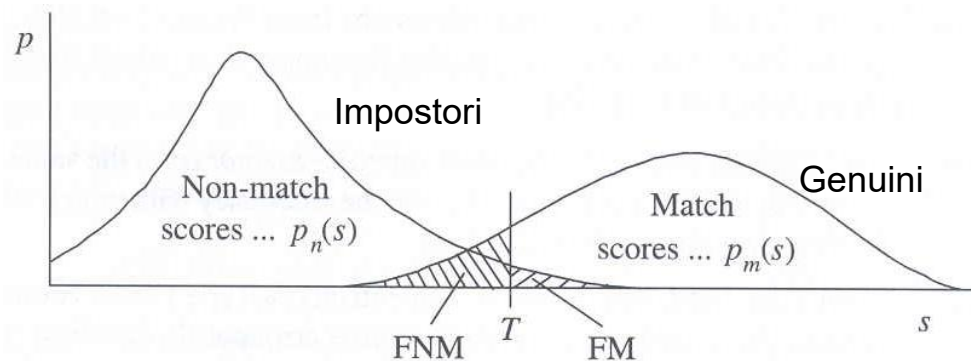
**SONO
DISTRIBUZIONI**

Dalle distribuzioni al FM Rate

- Come vediamo dal grafico il numero di persone appartenenti al gruppo dei **genuini** sono **sotto la soglia T** (FNR) deve essere calcolato come un integrale della porzione di curva dei genuini fino alla soglia T

$$FMR = \int_{s=T}^{\infty} \overset{\text{impostori}}{p_n(s)} ds = 1 - \int_{-\infty}^T p_n(s) ds$$

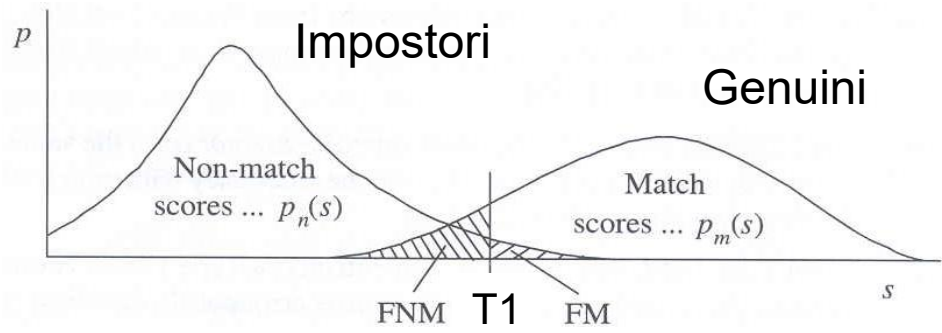
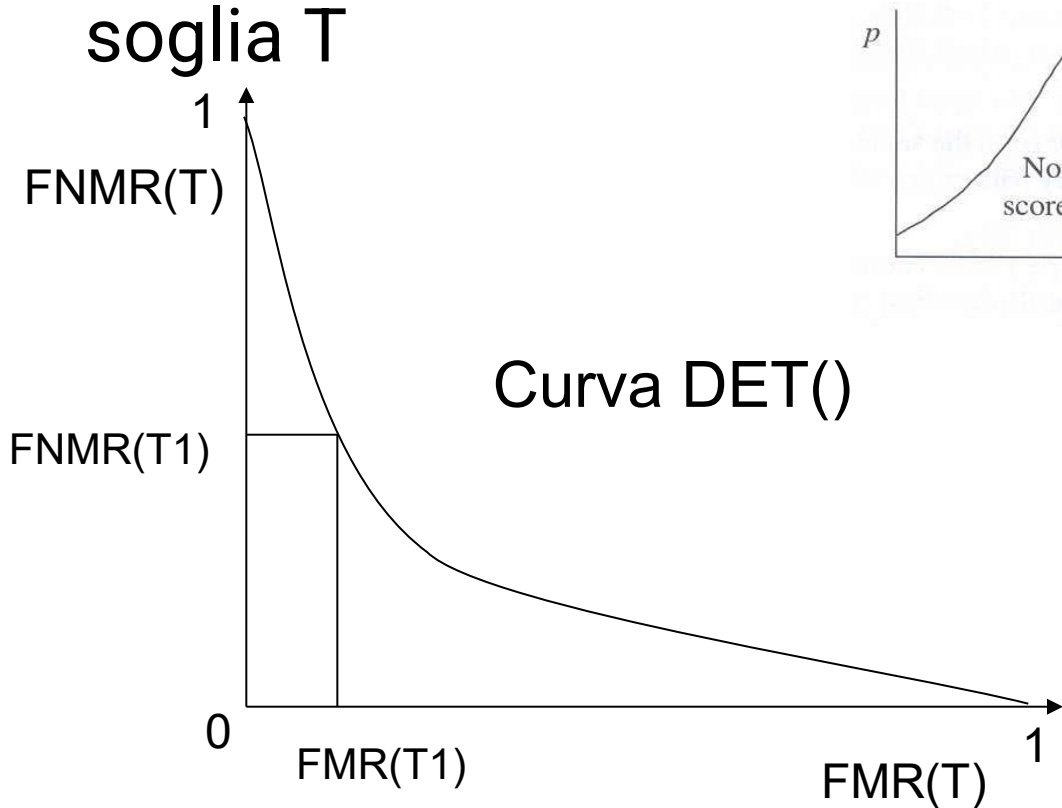
$$FNMR(T) = \int_{s=-\infty}^T \overset{\text{genuini}}{p_m(s)} ds$$



Queste formule si applicano solo se si conoscono le distribuzioni p_n e p_m altrimenti le approssimo con gli istogrammi e i sommatorie/conteggi

Decision Error Tradeoff (DET) Receiver Operating Characteristic (ROC)

Le performance del sistema vengono espresse mostrando come variano FMRT e FMR con la soglia T

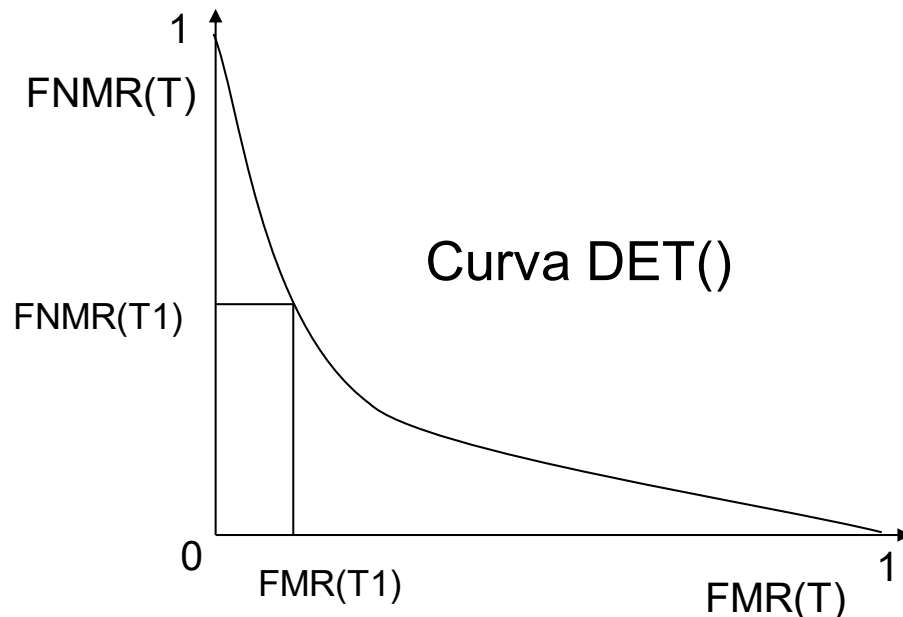


$$\begin{aligned} \text{DET}(T) &= (\text{FMR}(T), \text{FNMR}(T)) \\ \text{ROC}(T) &= 1 - \text{DET}(T) \\ \text{DET}(T) &\rightarrow \begin{cases} (1,0) & T \rightarrow -\infty \\ (0,1) & T \rightarrow +\infty \end{cases} \end{aligned}$$

DATA REPORTING

LA CURVA DET

- Secondo fondamentale dato da ricavare!
 - Controllare con esperimenti se disponibile dai datasheet....



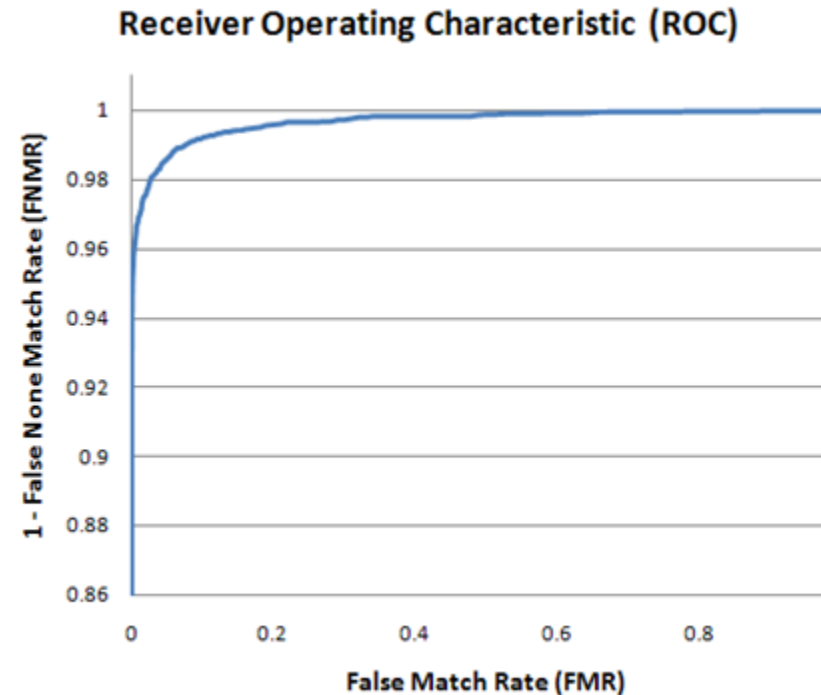
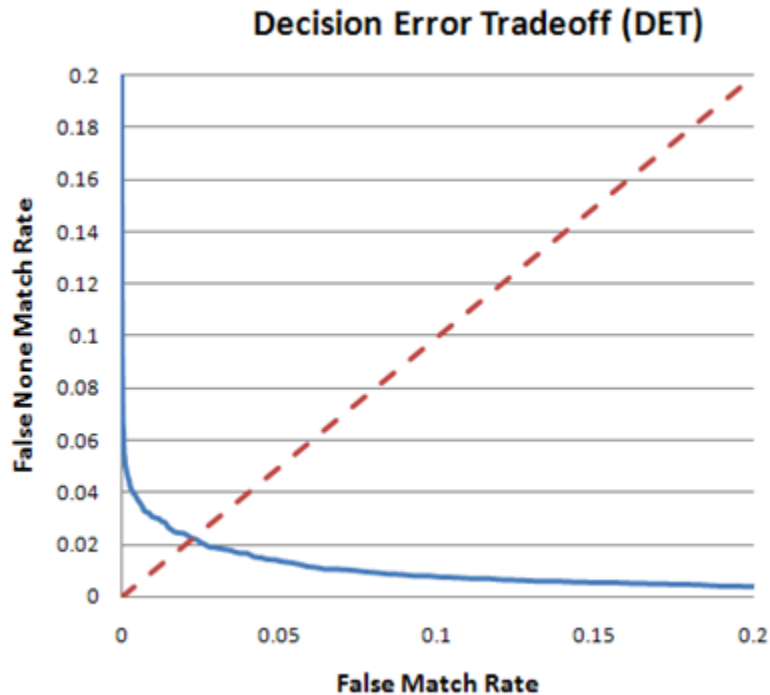
Cerco sempre di ottenerla da....

- interpolazione del datasheet
- dalla rivista scientifica
- dai miei dati personali precedenti
- da una valutazione
 - Technology,
 - Scenario,
 - Operational
- dagli esperimenti con i volontari

FMR, FNMR, FAR e FNAR

- Il funzionamento di un sistema biometrico dal punto di vista degli errori commessi è principalmente descritto dai tassi $FMR(T)$ e $FNMR(T)$ descritti per tutti i valori della soglia
- Nel caso si tratti di una identificazione positiva i tassi e vengono rispettivamente chiamati
 - $FMR(T) \rightarrow$ False Accept Rate $FAR(T)$
 - $FNMR(T) \rightarrow$
 - False Non-Accept Rate $FNAR(T)$
 - False Reject Rate $FRR(T)$
- Ricordiamo che $FMR(T)$ e $FNMR(T)$ sono in funzione della soglia T scelta

$$\text{DET} = 1 - \text{ROC}$$

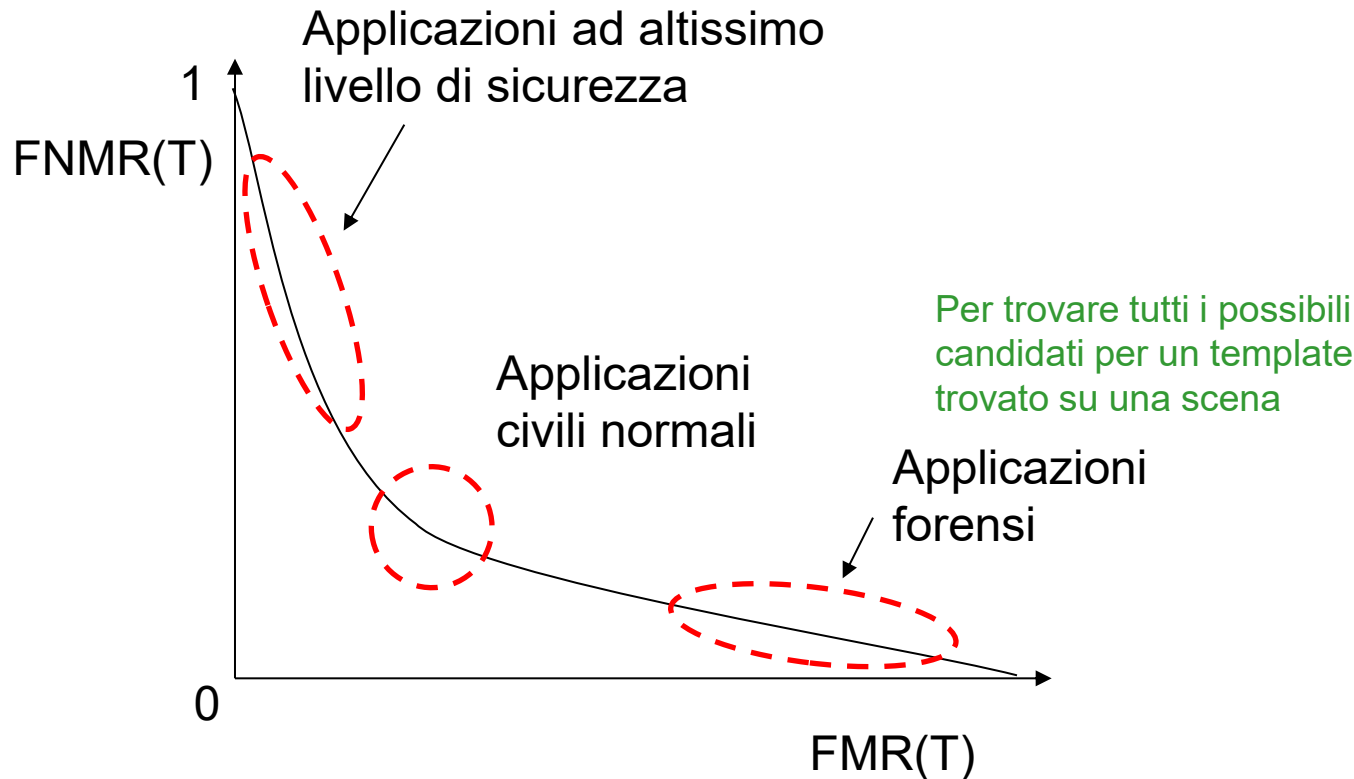


La curva DET e la curva ROC mostrano le stesse informazioni.
Riferendoci ad autenticazione positiva, la DET mette l'attenzione sul FNM (genuini che non entrano) mentre la ROC mette in evidenza 1-FNM (quanti genuini che riescono ad entrare)

Regioni di funzionamento

- Regolando la soglia T possiamo regolare il livello di sicurezza

Deposito di esplosivi



(*)
Attenzione nella dispensa corrispondente alla lezione successiva, gli assi sono invertiti.

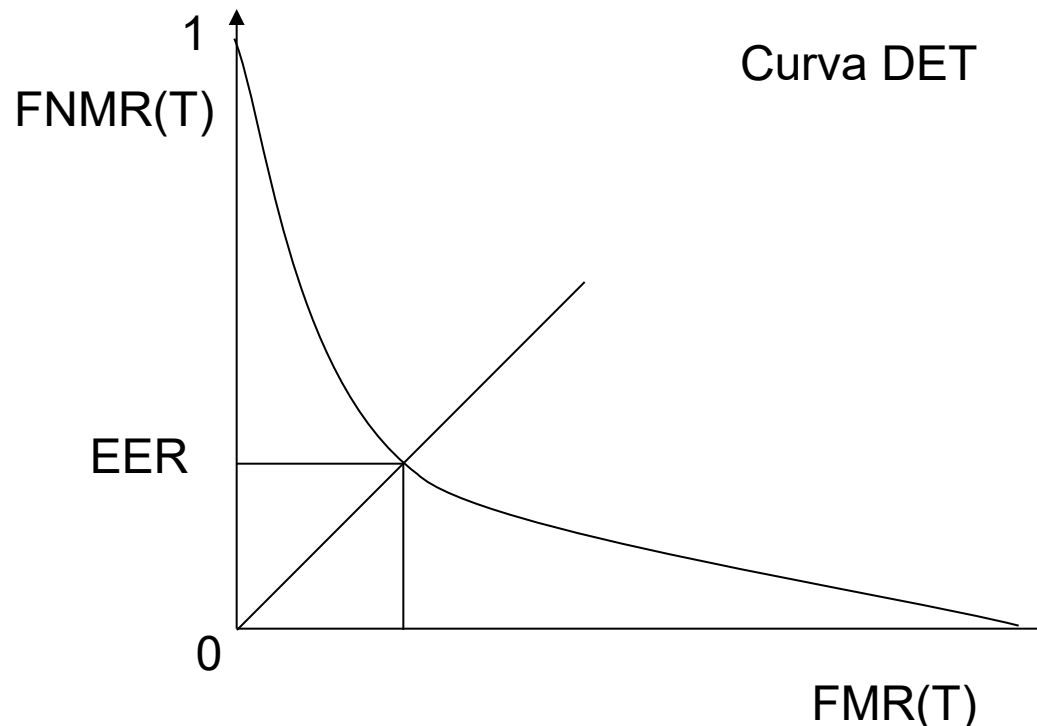
Prestare sempre la massima attenzione agli assi leggendo una curva DET!

Nota:
 $DET = (1 - ROC)$

Non si può regolare completamente a piacimento la soglia!

Equal Error Rate

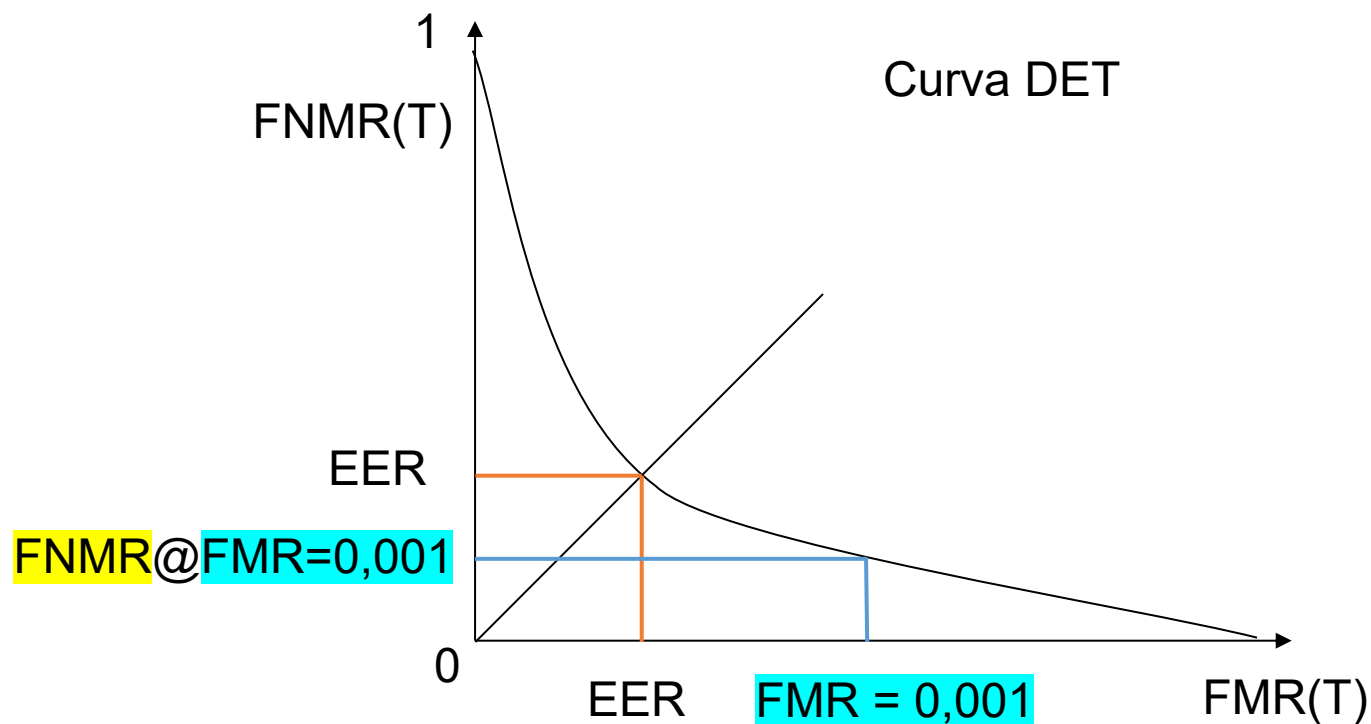
- L'EER è il tasso di errore corrispondente all'unico punto nel quale abbiamo $FNMR=FMR$
- L'EER è l'unico numero singolo che può riassumere il funzionamento del sistema

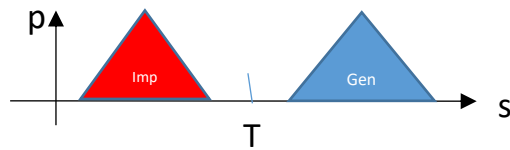


DATA REPORTING

EER o $\text{FNMR}@FMR=xxx$

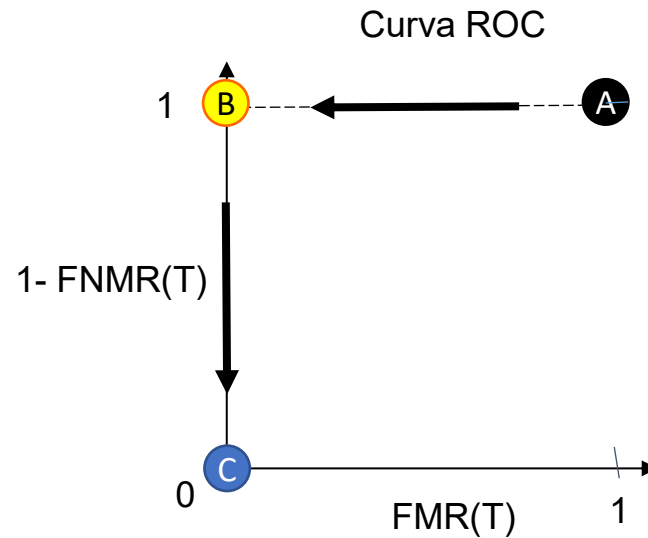
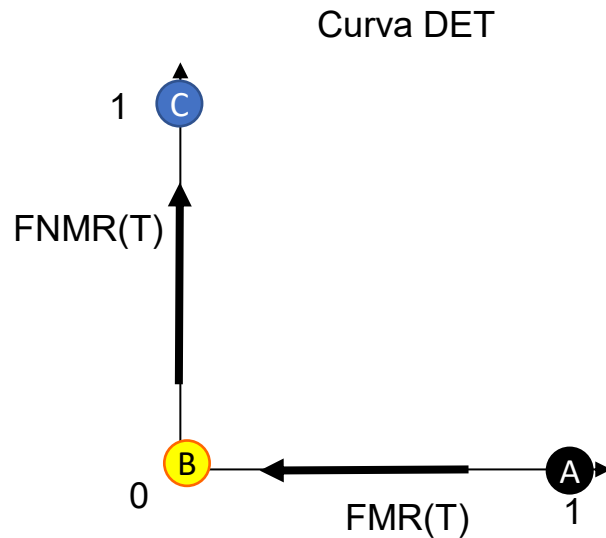
- Terzo fondamentale dato da ricavare!
 - Controllare con esperimenti se disponibile dai datasheet....





DET: Sistema ideale

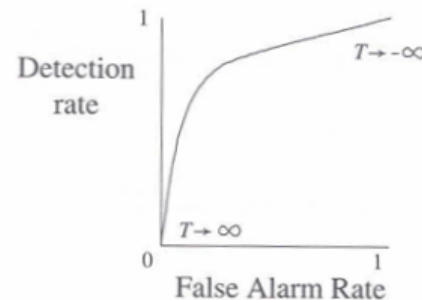
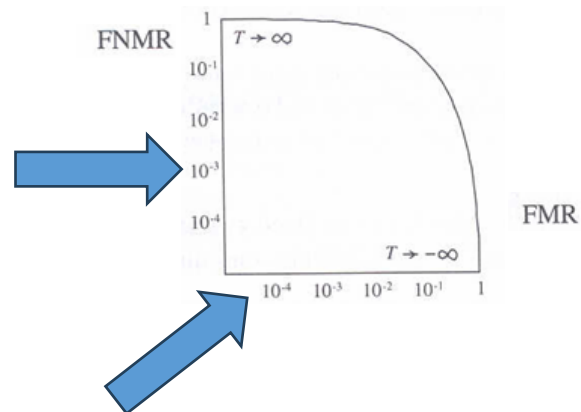
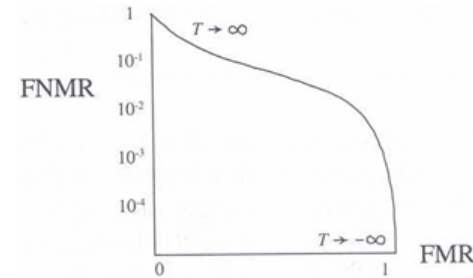
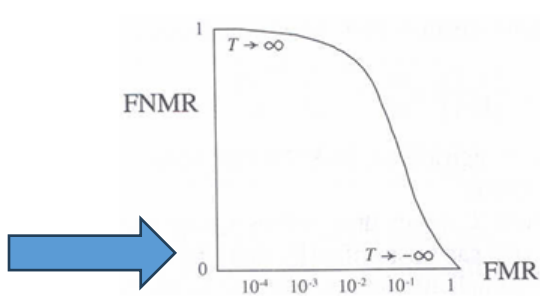
- Soglia $-\infty$: (FMR=1, FNMR=0) **A**
- Soglia intermedia : (FMR=0, FNMR=0) **B**
- Soglia $+\infty$: (FMR=0, FNMR=1) **C**



Variazioni della ROC

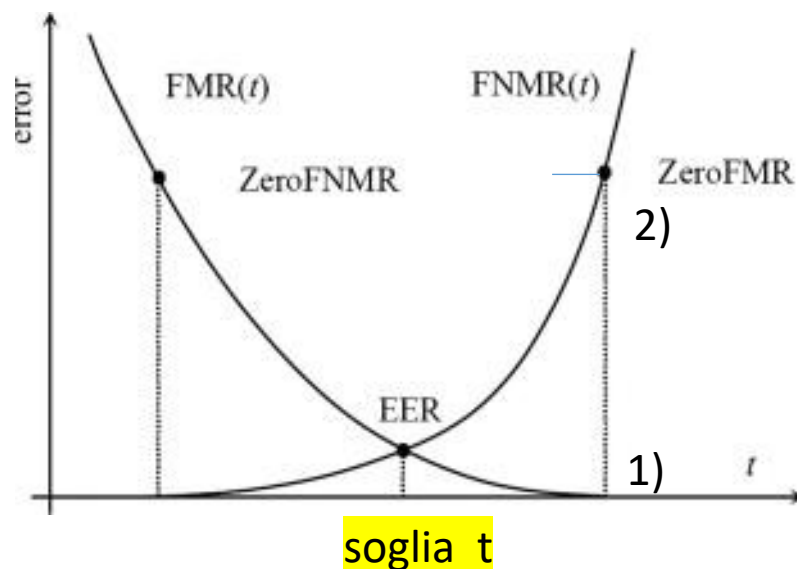
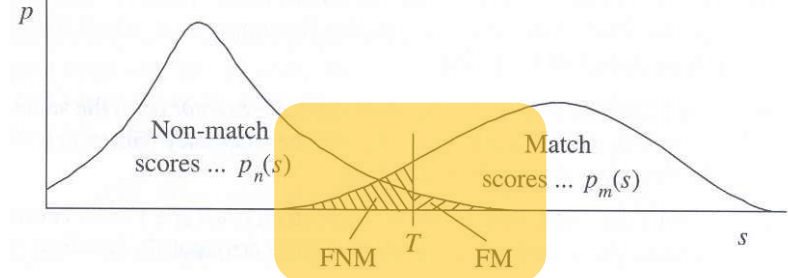
- Cambiando la scala degli assi si ottengono delle rappresentazioni equivalenti

Se l'errore su
una asse
diventa molto
piccolo
meglio
mettere l'asse
in scala
logaritmica



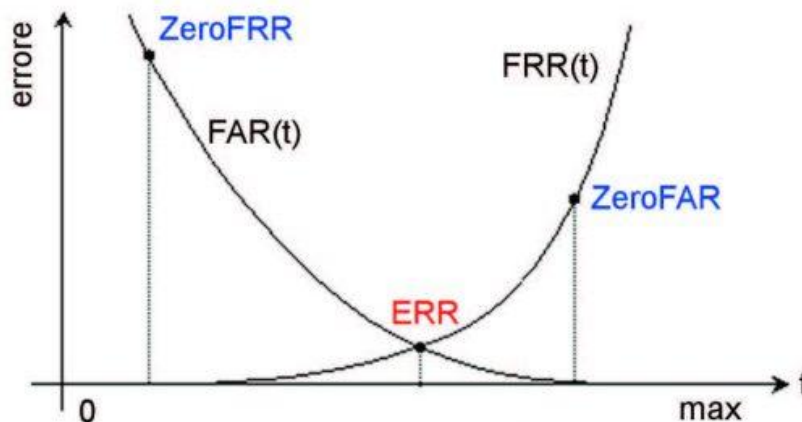
ZeroFMR e ZeroFNMR

- Come possiamo capire quanto alzare la soglia della somiglianza per non far entrare nessun impostore?
- E quanti genuini rimangono fuori?
- ZeroFMR e' la frazione di genuini ($FNMR(t)$) sotto soglia che corrisponde ad avere $FMR = \text{zero}$.
- E viceversa



ZeroFAR e ZeroFRR

Per quanto i termini FMR e FNMR valgono sempre, talvolta nei sistemi di riconoscimento positive si parla di FAR e FRR rispettivamente



ZeroFAR = FRR ottenuto per il valore della soglia minimo tale che il FAR sia nullo.”

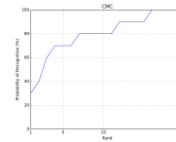
ZeroFRR = FAR ottenuto per il valore della soglia massimo tale che il FRR sia nullo”

La regolazione della soglia di un Sistema biometrico al valore di soglia corrispondente a

- zeroFMR (zeroFAR) corrisponde al funzionamento ad alta **security**
- zeroFNMR (zeroFRR) corrisponde al funzionamento ad alta **convenience**

Esempio di creazione di CMC

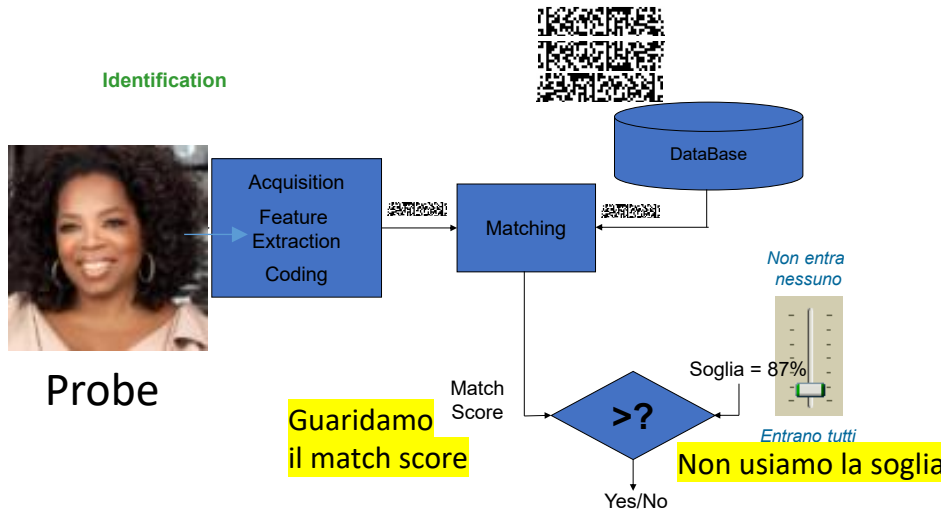
k = 1



k = 1 (più simile)

(Sistema perfezionabile come identificatore

Numero persone in Gallery = 5



Score = 0.71
Errore!



Score = 0.70



Score = 0.32



Score = 0.30



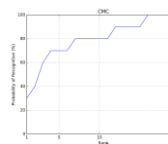
Score = 0.23

Mettiamo 1 immagine in ingresso come Probe.
Avendo 5 persone in Gallery per procedere alla identificazione potremo fare 5 comparazioni.
Mettiamo gli esiti del matching in ordine di similitudine.
Supponiamo che ci venga chiesto una analisi con k=1
Prendiamo il più somigliante e vediamo se abbiamo un errore. In questo caso abbiamo un errore.

Ripetiamo e contiamo gli errori per tutte le persone della Gallery, ottenendo il tasso di errore della CMC(k=1)

Esempio di creazione di CMC

$k = 2$



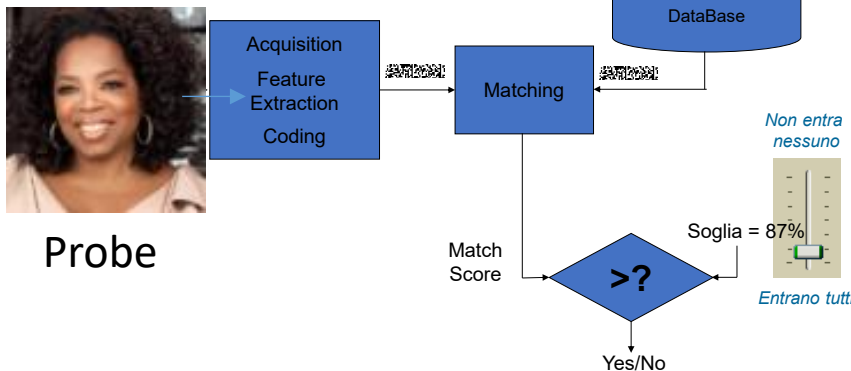
$k = 2$ (se è fra i 2 più simili)

(forse però come sistema di screening funziona!)

Numero persone in Gallery = 5



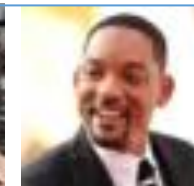
Identification



Score = 0.71



Score = 0.70
Corretto!



Score = 0.32



Score = 0.30



Score = 0.23

Mettiamo 1 immagine in ingresso come Probe.
Avendo 5 persone in Gallery per procedere alla identificazione
potremo fare 5 comparazioni.

Mettiamo gli esiti del matching in ordine di similitudine.
Ora dobbiamo controllare se l'abbiamo identificata nel gruppo
di 2 persone ($k=2$) prendendo i 2 più somiglianti. In questo
caso il soggetto è dentro il gruppo dei primi k con $k=2$ quindi
abbiamo un successo.

**Ripetiamo e contiamo gli errori per tutte le persone della
Gallery ottenendo il tasso di errore della CMC($k=2$).**

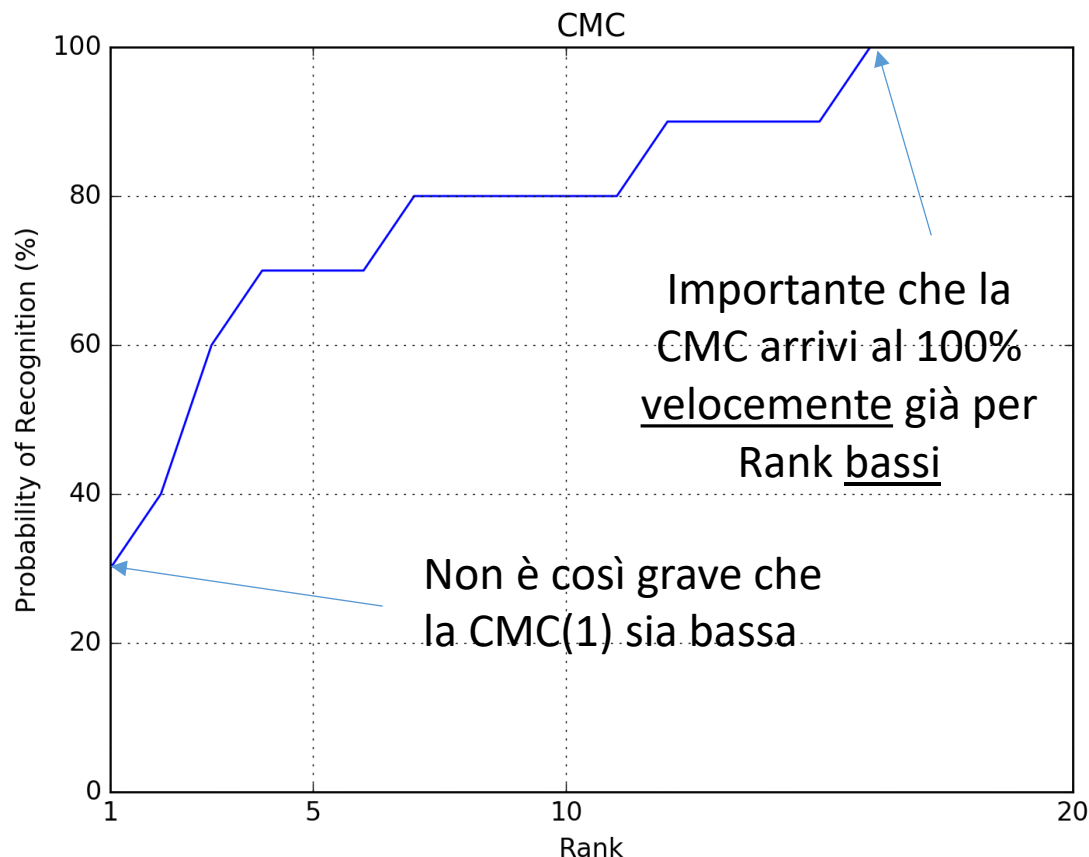
Procedere per gli altri K

Più grande il k , maggiore la probabilità di avere nell'insieme
di k persone la persona che stiamo cercando di individuare.

36

Accuratezza in identificazione Cumulative Match Characteristic (CMC)

- Nel contesto della **identificazione** si può valutare il grafico CMC(R)



PROCEDIMENTO DI CALCOLO

Siano N le persone in nel DB

Per ogni individuo k ordino

gli R (=rank) individui

più simili a k .

Conto quante volte k era

negli R trovati.

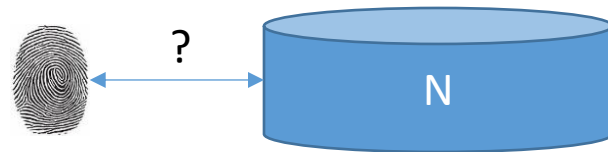
Ripeto per tutti k possibili

da $1:N$

DATA REPORTING

CMC(R)

Identificazione: metodo gerarchico



Possibile soluzione per un problema di
identificazione con N grande:
uso due sistemi biometrici in cascata

→ **Sis.Biom. #1 (SB1)**

Sistema adatto alla identificazione
con N grande (iride, impronta) per
velocità e memoria

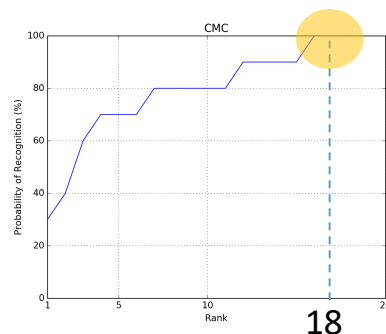
→ Guardo CMC e **scelgo Rank K** con
probabilità di recognition del 100%

→ Riduco a K (pochi) candidati

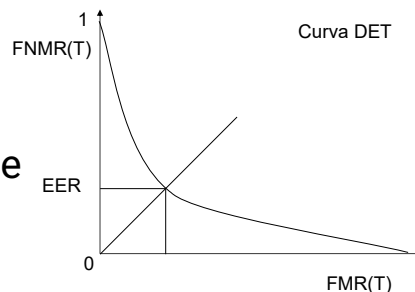
→ **Sis.Biom. #2 (SB2)**

Sistema accurato con ottima DET
(magari inefficiente per identificazione),
fisso una soglia T e

- scelgo dei K candidati quello più somigliante
(se sopra soglia T)
- altrimenti out = non trovato

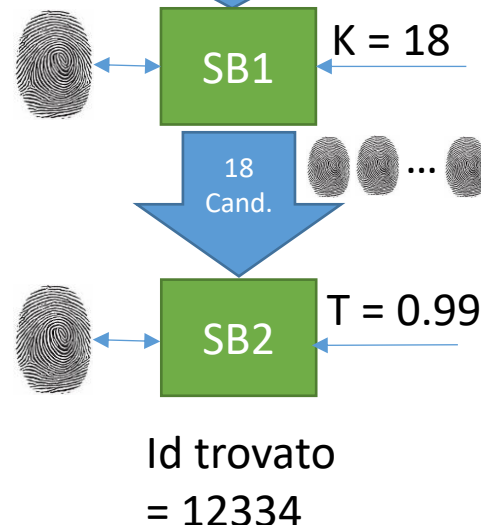


18



Esempio

1.000.000
Candidati



Fissare la soglia T , Calcolare p

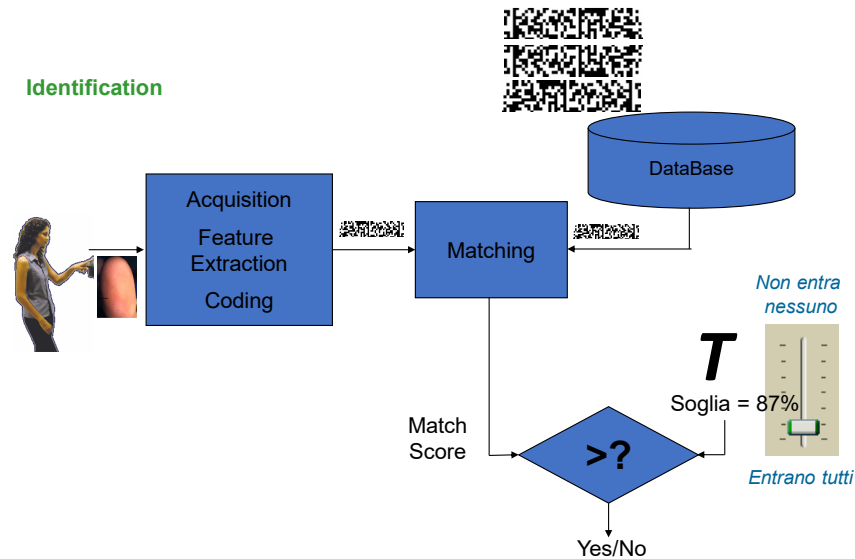
Una volta fissata
la soglia T (*similitudine**)
il sistema è compl. Definito
→ Diventa un classificatore

Regola di comportamento del sistema

$$(I, X_Q) \in \begin{cases} \omega_1 & \text{se } S(X_Q, X_I) \geq T \\ \omega_2 & \text{altrimenti} \end{cases}$$

Si definisce quindi la probabilità di errore
 $p = P(\text{errore}) = P(FM(T)) + P(FNM(T))$

Stima possibile $p = (FM + FNM) / \text{TOTALE}$

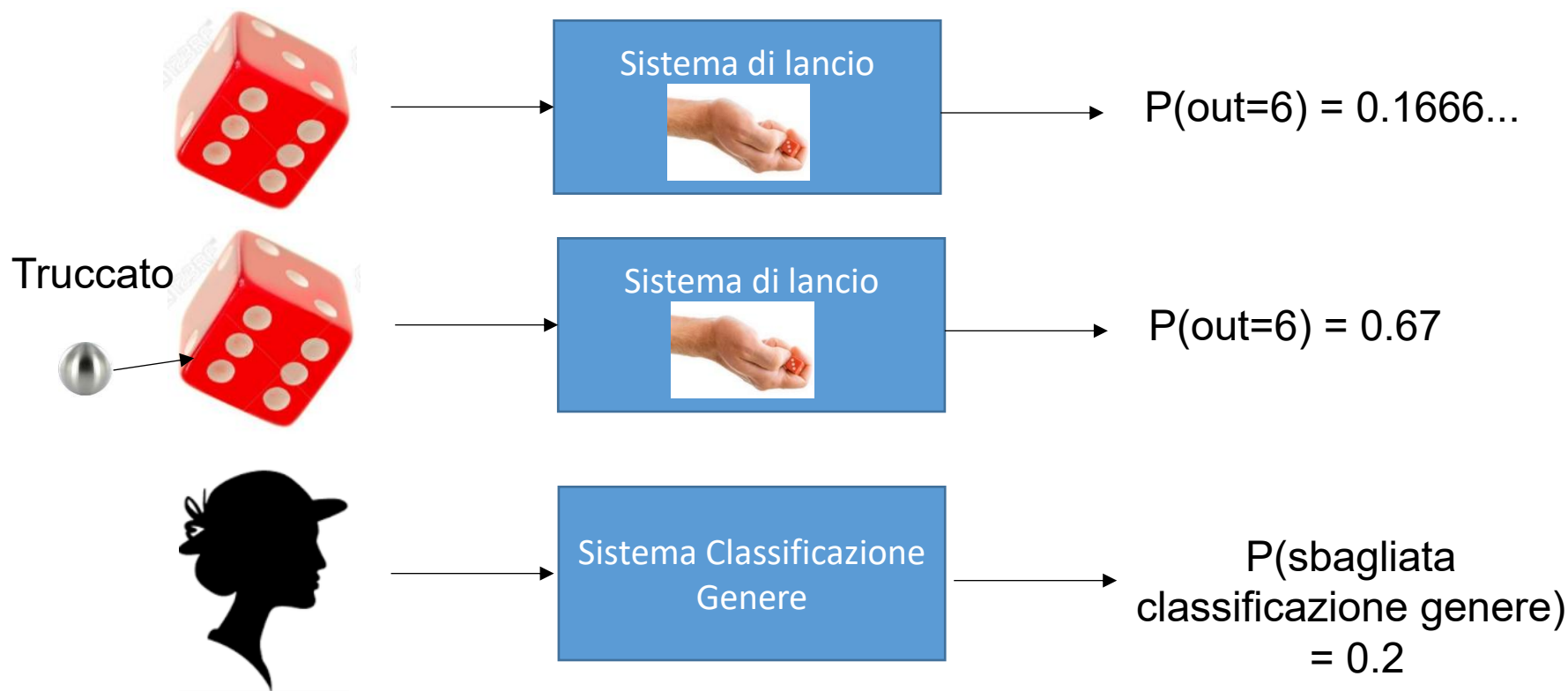


DATA REPORTING

Che modello usiamo?

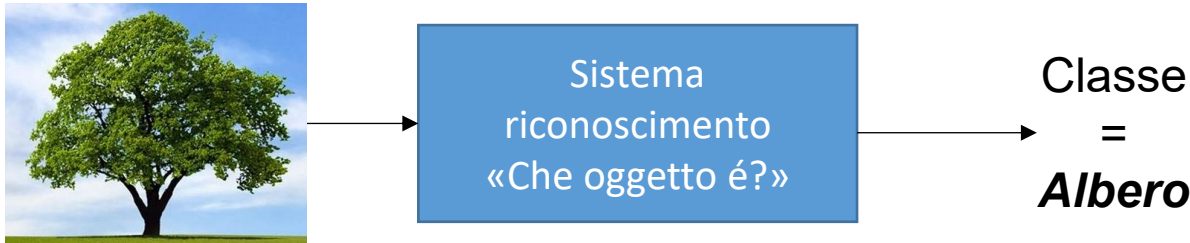
Quale è il modello statistico migliore per caratterizzare un Sistema Biometrico (**SB**)?

Facciamo alcuni esempi per introdurre il discorso

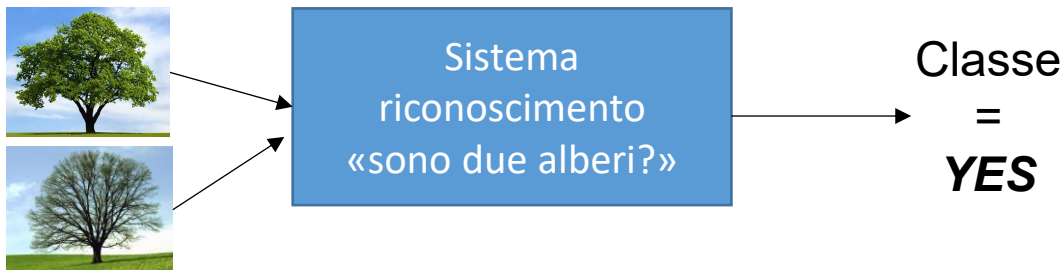
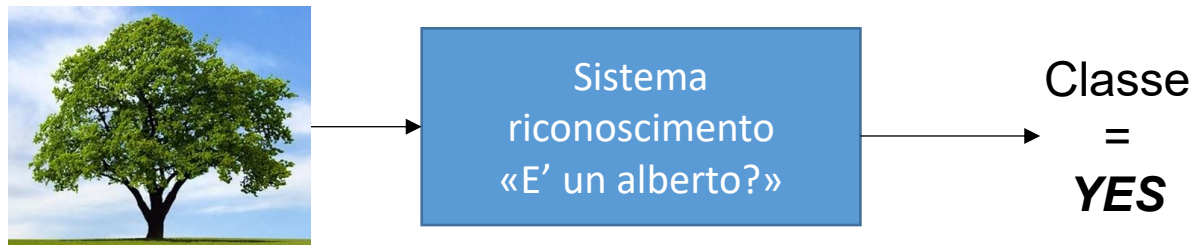


Tipico classifikatore di immagini

- Multiclasse

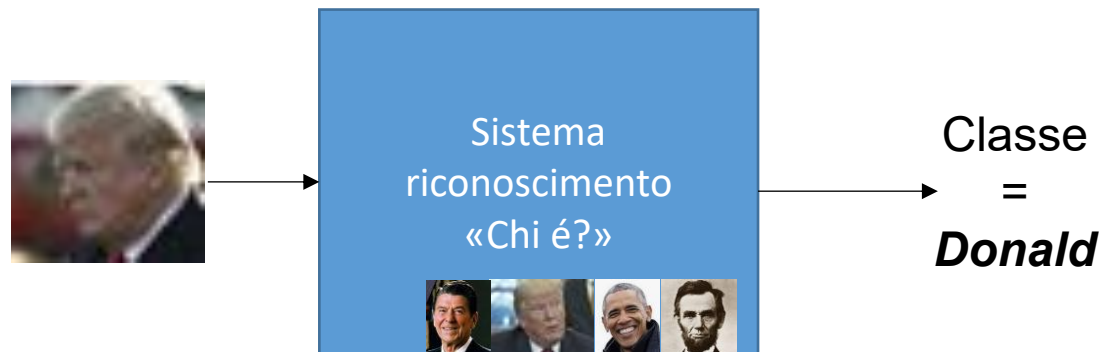


- Binario

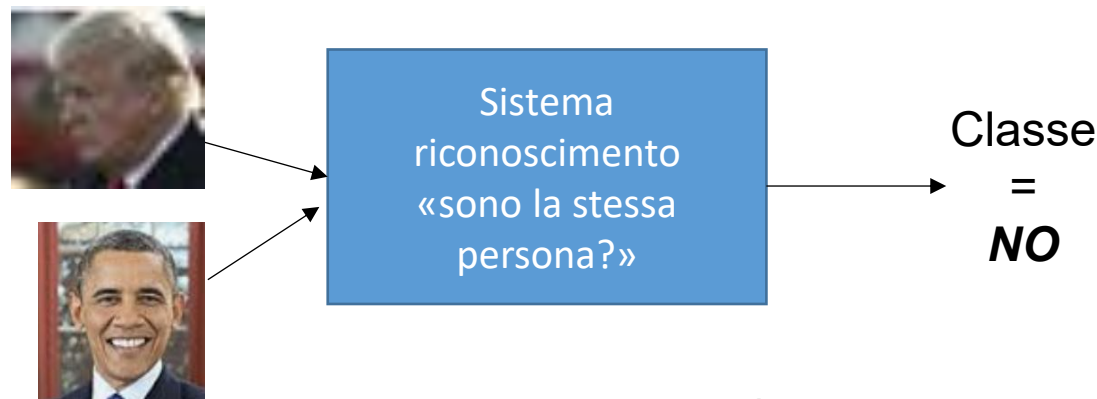


Classificatore biometrico

- Multiclasse = Identificatore

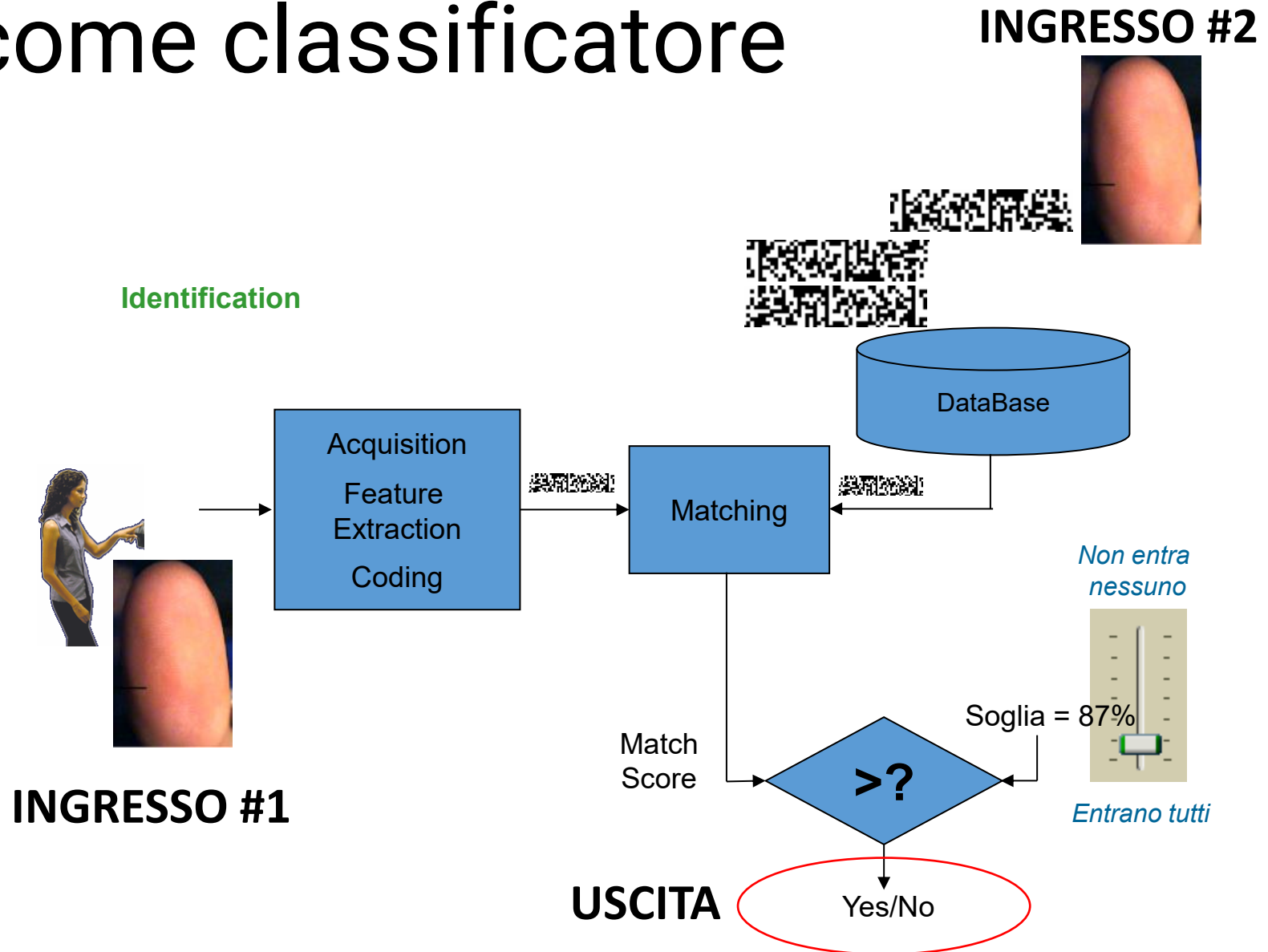


- Binario = Autenticatore

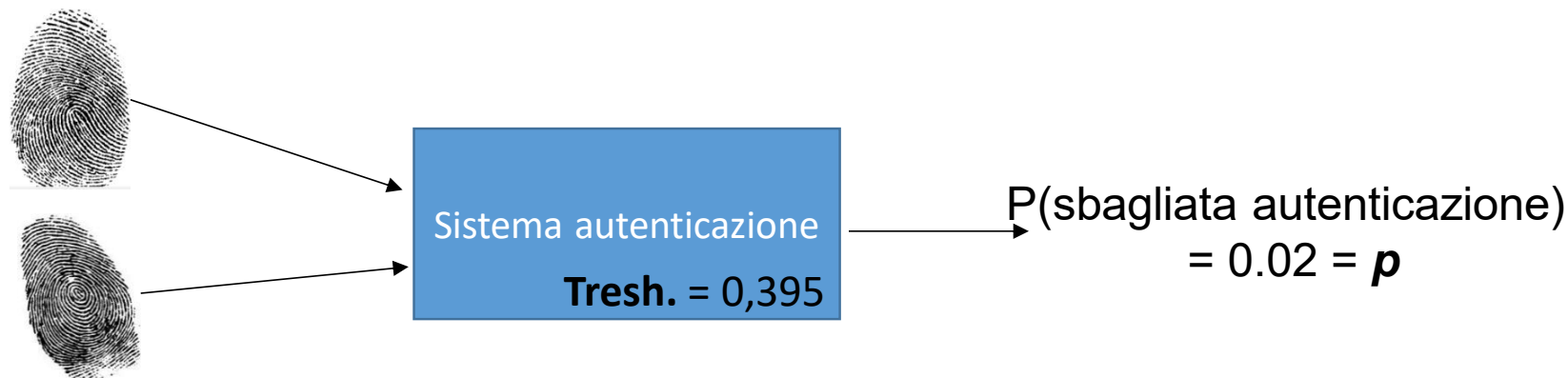
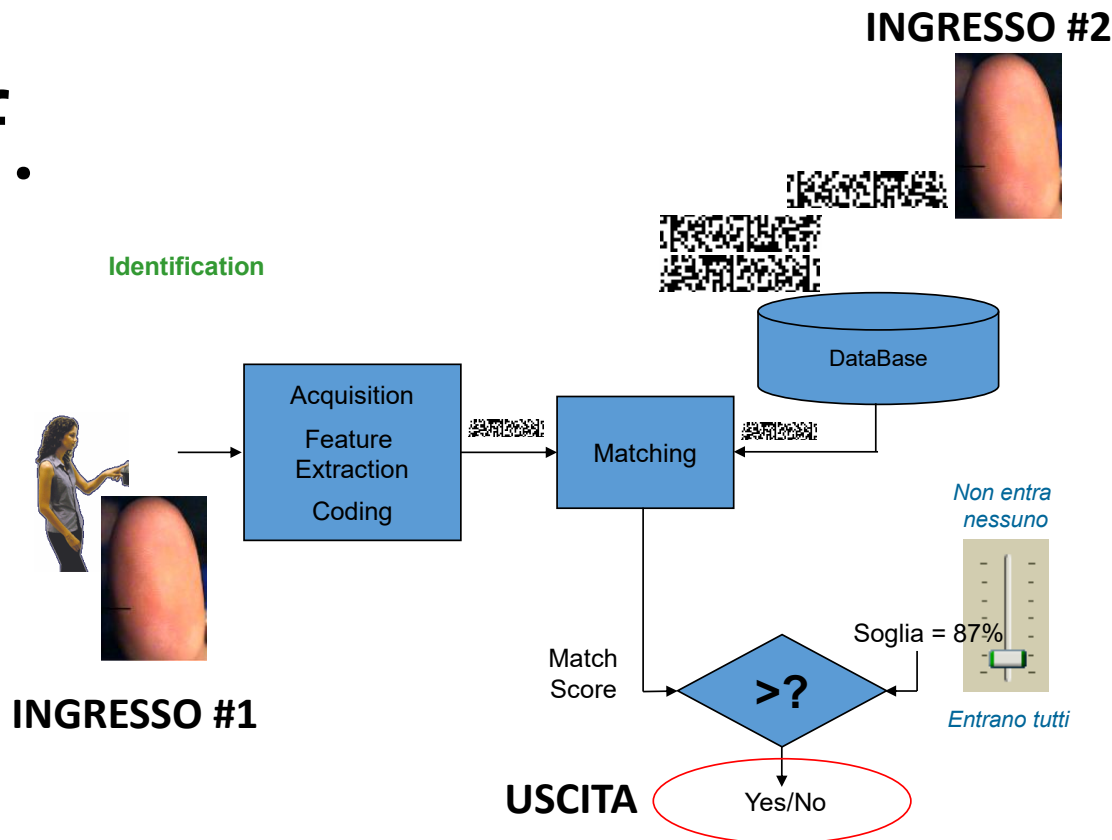


Sistema
biometrico
=
classificatore
 $P(\text{errore}) = p$

Il sistema biometrico visto come classificatore

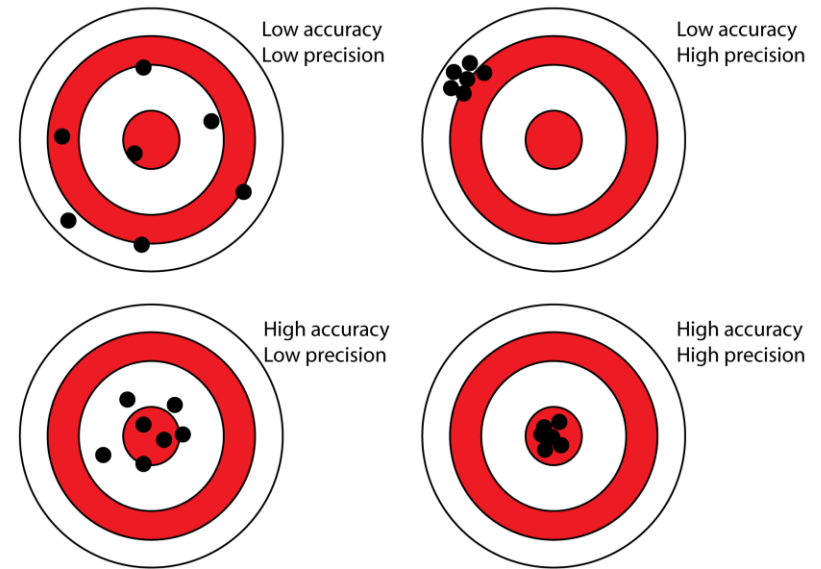


SB = classif.



Valutazione

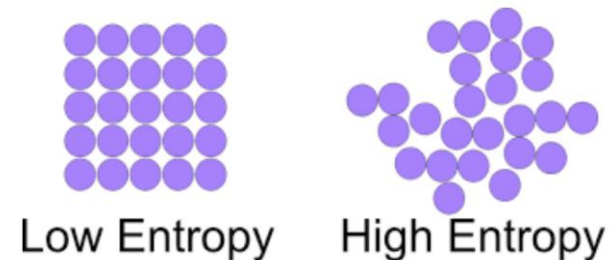
- Accuratezza
- Precisione e Recall
- Squared error
- Likelihood
- Posterior probability
- Cost / Utility
- Margin
- Entropy
- K-L divergence
- Ecc.

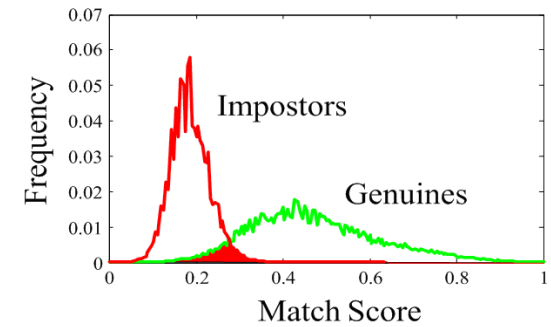


$$P(c|x) = \frac{P(x|c)P(c)}{P(x)}$$

Diagram illustrating the relationship between probabilities in a classification model:

- $P(c|x)$ is labeled as **Posterior Probability**.
- $P(x|c)$ is labeled as **Likelihood**.
- $P(c)$ is labeled as **Class Prior Probability**.
- $P(x)$ is labeled as **Predictor Prior Probability**.





Reti Deep per il volto e Modelli e metodi statistici per la biometria

Fabio Scotti

fabio.scotti@unimi.it

Università degli Studi di Milano



Parte B

Approfondimento

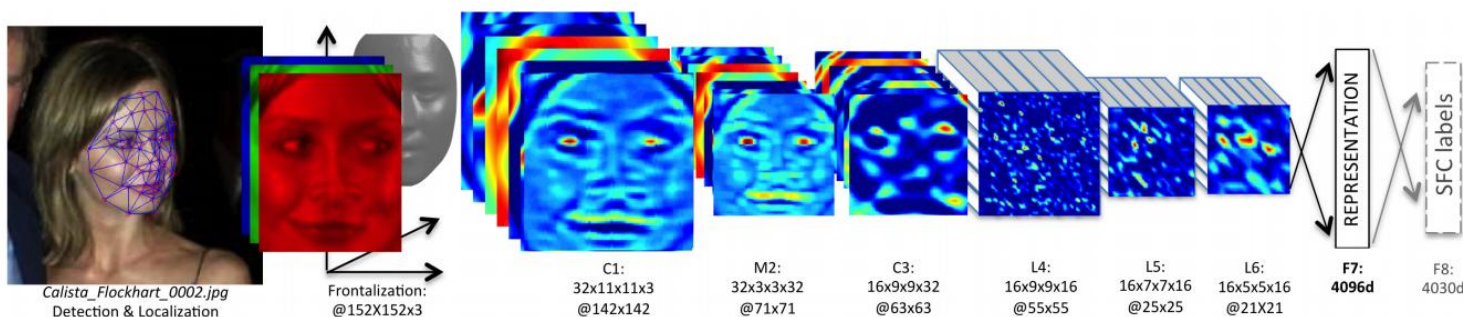
Reti Deep per il volto

Fabio Scotti
fabio.scotti@unimi.it

Università degli Studi di Milano

DeepFace

- Andiamo a dettagliare meglio una delle reti maggiormente versatili, utili e aggiornate a disposizione
 - Cosa è DeepFace?
 - Quale è la sua importanza nel campo della Computer Vision e dell'Intelligenza Artificiale e dei sistemi biometrici avanzati



Contesto e Panoramica delle funzioni

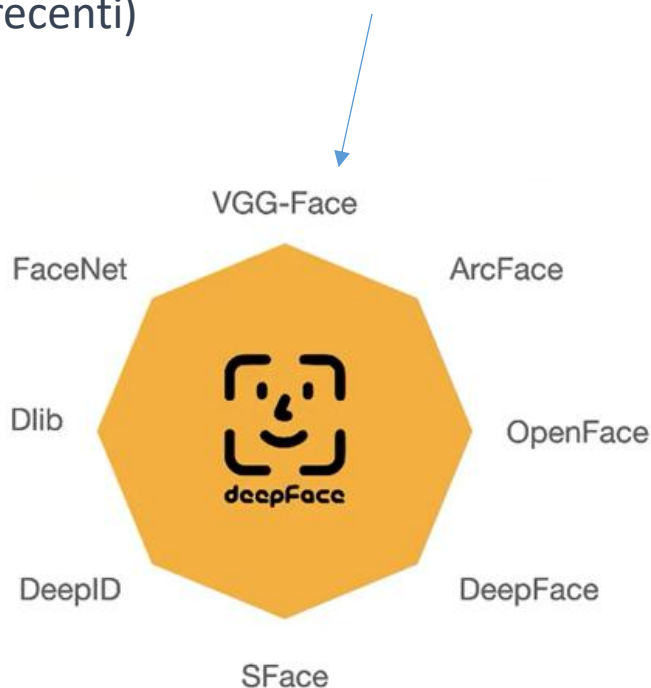
- Il codice di DeepFace (scritto principalmente da Sefik Ilkin Serengil) non nasce da zero, ma integra e riusa (wrapping + adattamento) implementazioni open-source
- Libreria Python open-source (<https://pypi.org/project/deepface/>)
- Interfaccia unificata per il riconoscimento facciale basato su deep learning,
- Integra modelli SOTA (es. VGG-Face, Facenet, ArcFace) e pipeline complete di face detection, embedding e matching.

FUNZIONI DISPONIBILI

- **Face Verification**
- **Face Identification**
- **Analisi degli Attributi Facciali (Facial Attribute Analysis)**
 - descrizione delle proprietà visive delle immagini del volto come età, classificazione del genere, analisi delle emozioni o previsione di razza/etnia.
- **Analisi del Volto in Tempo Reale (Real-Time Face Analysis)**
 - riconoscimento del volto e dell'analisi degli attributi facciali con il flusso video in tempo reale per esempio da una webcam

Deepface contiene diversi modelli (chiamarlo «wrapper» è riduttivo...)

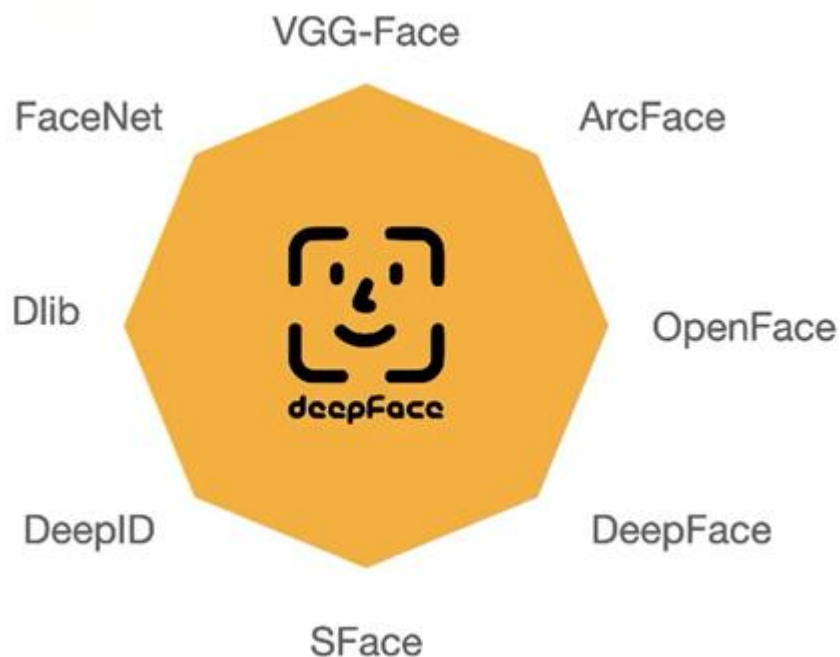
VGG-Face: questo modello è noto per la sua profondità e ha mostrato buone prestazioni in vari task di visione artificiale. È stato adattato per il riconoscimento facciale da un modello VGG pre-addestrato (vedi lezioni precedenti)



Il modello di riconoscimento facciale VGG raggiunge una precisione del 97,78% sul popolare dataset Labeled Faces in the Wild (LFW).

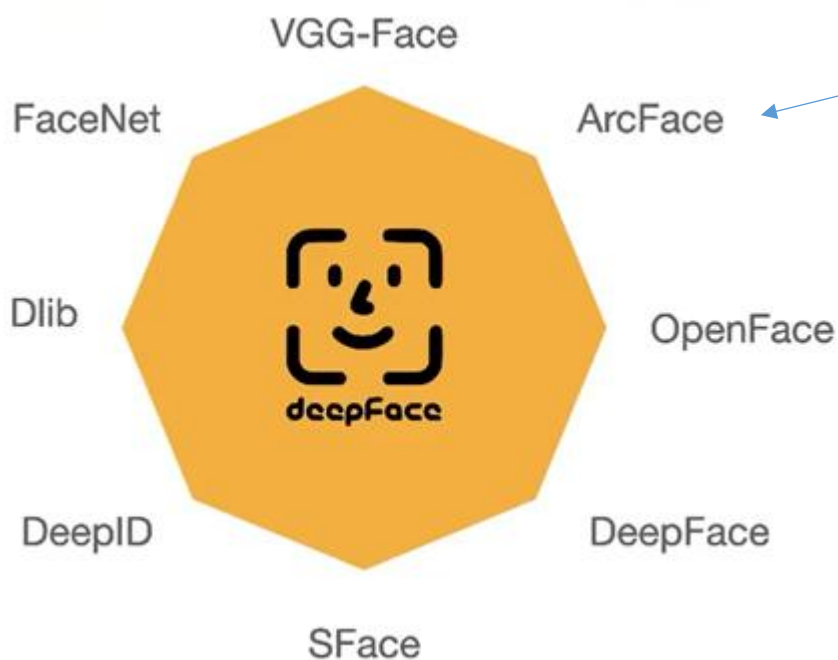
la libreria DeepFace utilizza VGG-Face come modello predefinito.

Deepface contiene diversi modelli



OpenFace è un modello open-source che mira a fornire prestazioni comparabili a FaceNet con una complessità computazionale inferiore. È adatto per applicazioni in tempo reale.

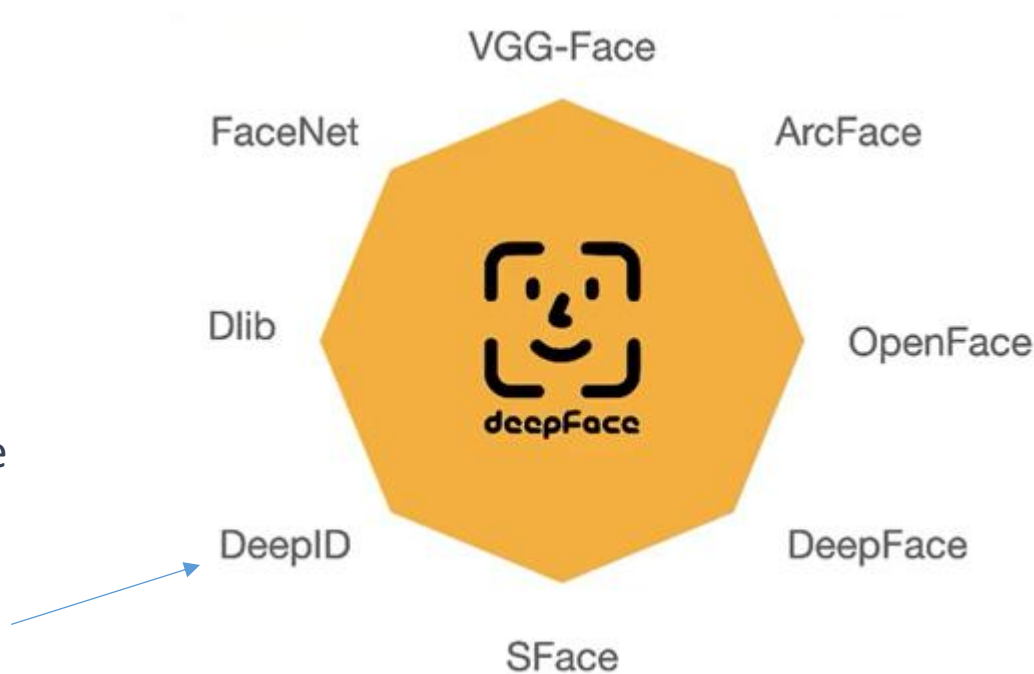
Deepface contiene diversi modelli



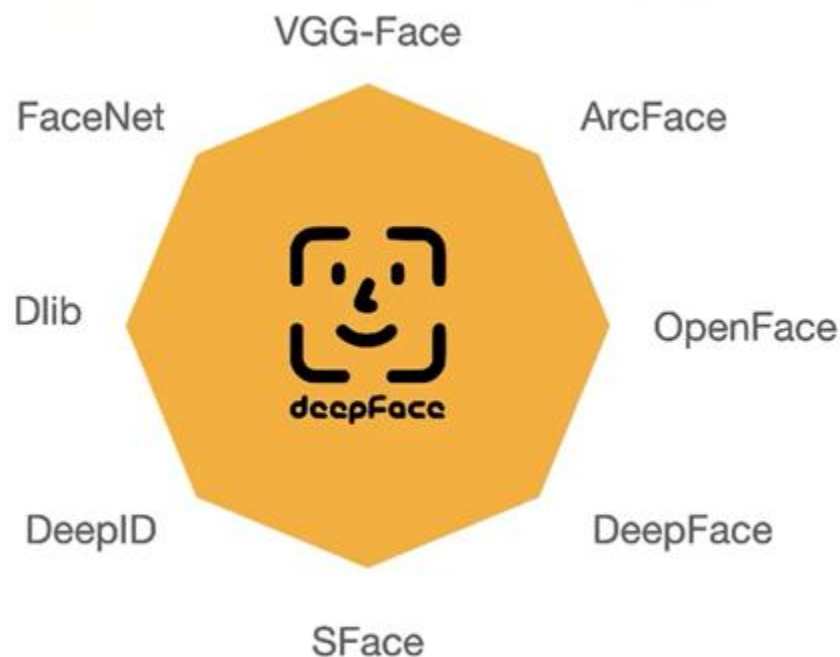
ArcFace è noto per la sua robustezza e accuratezza, specialmente in scenari con classi aperte, dove il numero di identità è sconosciuto durante la fase di addestramento (es. videosurveillance)

Deepface contiene diversi modelli

DeepID è una serie di modelli che sono stati sviluppati nel tempo, con miglioramenti incrementali. Sono noti per essere efficaci nel gestire variazioni come angolazioni, illuminazione e espressioni.



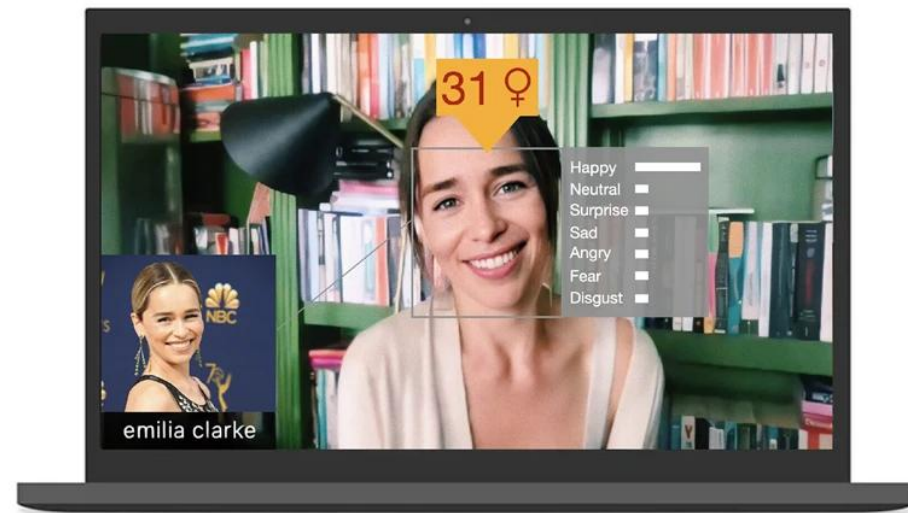
Deepface contiene diversi modelli



Dlib: denominato «La più semplice API di riconoscimento facciale al mondo per Python»

- Mappa un'immagine di un volto umano in uno spazio vettoriale a 128 dimensioni
- Il matching è una distanza euclidea!

Come scelgo?



```
DeepFace.stream(db_path = "C:/User/Sefik/Desktop/database")
```

La funzione "Stream" accederà alla tua webcam e applicherà sia il riconoscimento del volto che l'analisi degli attributi facciali.

- Quando scegliete un modello da utilizzare o allenare, considerate vari fattori come l'accuratezza, la velocità di inferenza, e le risorse computazionali richieste.
- Ogni modello ha i suoi vantaggi e svantaggi in base al contesto applicativo specifico

Facial Recognition (detector_backend)

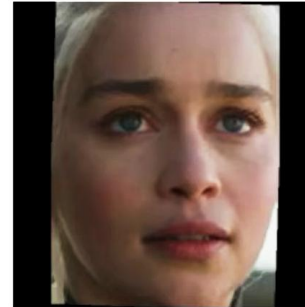
- Si hanno diversi modelli usabili per questo task



Original Image



RetinaFace



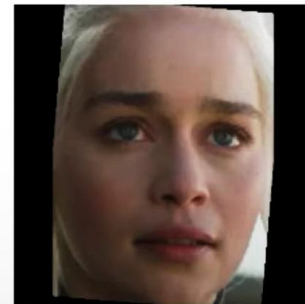
MtCnn



Dlib



MediaPipe



Ssd



OpenCv

Facial Recognition

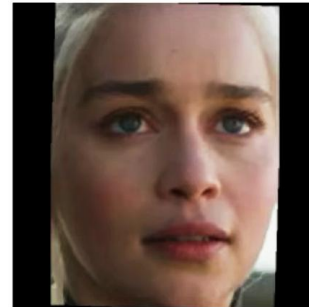
- Si hanno diversi modelli usabili per questo task



Original Image



RetinaFace



MtCnn



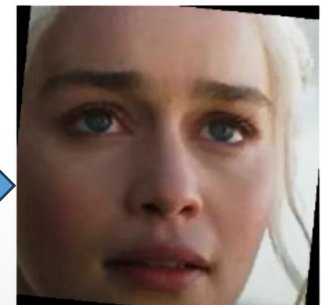
Dlib



MediaPipe



Ssd



OpenCv

Questo di OpenCV
lo conosciamo già

Facial Recognition

(Note utili in sviluppo)

Dlib

Questo rilevatore utilizza un algoritmo HOG (Histogram of Oriented Gradients) in background. Pertanto, similmente a OpenCV, non si basa sull'apprendimento profondo. Tuttavia, ha punteggi relativamente elevati per quanto riguarda il rilevamento e l'allineamento dei volti.

SSD

SSD sta per Single-Shot Detector; è un rilevatore basato sull'apprendimento profondo molto popolare. Le prestazioni di SSD sono paragonabili a quelle di OpenCV. Tuttavia, SSD non supporta i landmark facciali e dipende dal modulo di rilevamento degli occhi di OpenCV per l'allineamento. Anche se le sue prestazioni di rilevamento sono elevate, il punteggio di allineamento è solo medio.

MTCNN (Deep con landmark facciali)

Si tratta di un rilevatore di volti basato sull'apprendimento profondo, e include i landmark facciali. Questa è la ragione per cui sia i punteggi di rilevamento che di allineamento sono elevati per MTCNN. Tuttavia, è più lento rispetto a OpenCV, SSD e Dlib.

RetinaFace (il migliore ma lento)

RetinaFace è riconosciuto come il modello (pubblico) basato sull'apprendimento profondo più avanzato per il rilevamento dei volti. Le sue prestazioni "in the wild" (in condizioni non controllate) sono impegnative. Tuttavia, richiede un elevato potere computazionale. Ecco perché RetinaFace è il rilevatore di volti più lento in confronto agli altri.

RetinaFace....

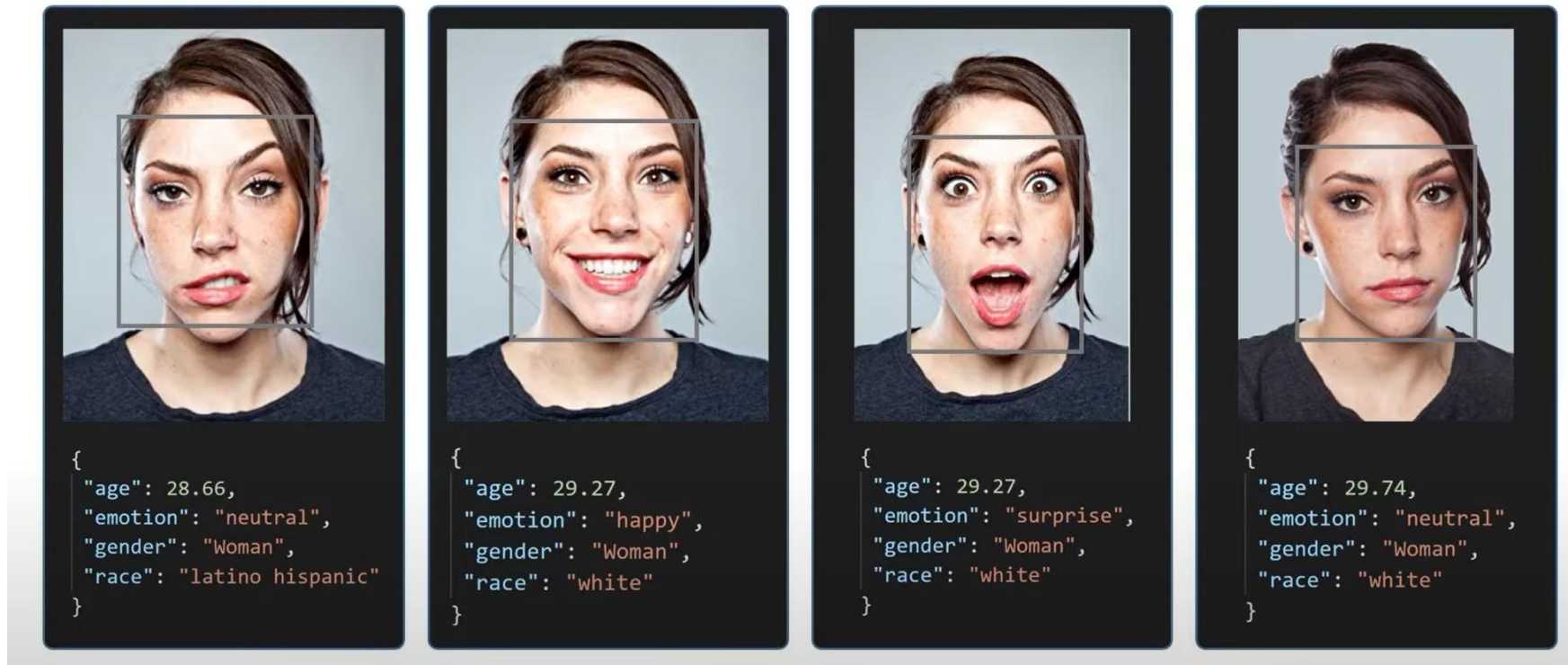


Uso molto intuitivo

```
detectors =  
["opencv", "ssd", "mtcnn", "dlib", "retinaface"]
```

```
img = DeepFace.detectFace("img1.jpg",  
detector_backend = detectors[4])
```

Emotion Detection



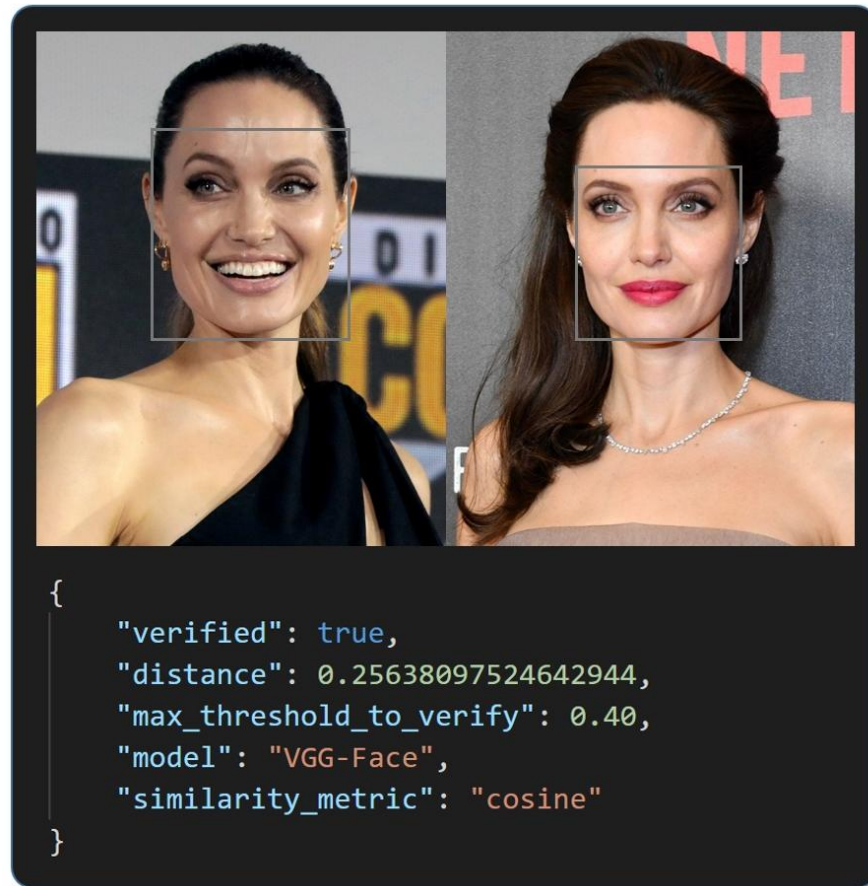
Più avanti nell'insegnamento lo tratteremo più in dettaglio

Alcune delle informazioni estraibili



```
{
  "emotion":{
    "angry":7.603101671639384e-14,
    "disgust":2.7474185705216866e-21,
    "fear":1.688688161735822e-14,
    "happy":100.0,
    "sad":4.205067717644173e-10,
    "surprise":7.103817571484745e-13,
    "neutral":4.4851553027136504e-08
  },
  "dominant_emotion":"happy",
  "age":31,
  "gender":"Woman",
  "race":{
    "asian":0.9087088517844677,
    "indian":1.1444833129644394,
    "black":0.09399998234584928,
    "white":66.56872034072876,
    "middle eastern":16.655877232551575,
    "latino hispanic":14.628209173679352
  },
  "dominant_race":"white"
}
```

Face authentication



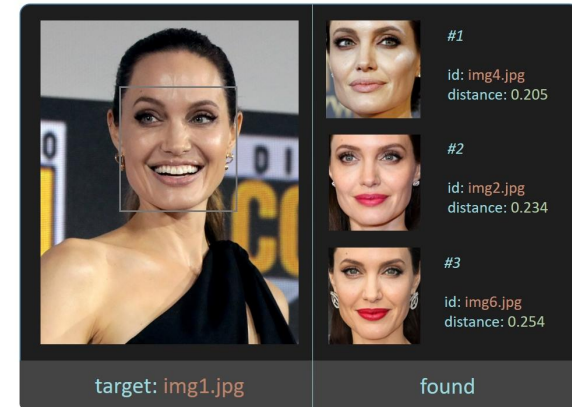
Face recognition (identification)

 target: <code>img1.jpg</code>	 #1 id: <code>img4.jpg</code> distance: 0.205  #2 id: <code>img2.jpg</code> distance: 0.234  #3 id: <code>img6.jpg</code> distance: 0.254 found
PROBE	GALLERY

Probe e Gallery

Sonda (Probe)

si riferisce al campione biometrico che viene attualmente interrogato o testato. Questo è il campione di dati che si desidera identificare o verificare. Ad esempio, in un sistema di riconoscimento facciale, la sonda potrebbe essere l'immagine del volto di una persona che si desidera identificare o verificare rispetto a un database. La sonda viene confrontata con la galleria per trovare una corrispondenza o per confermare un'identità.

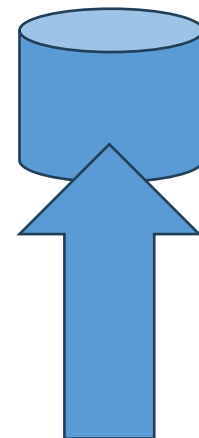
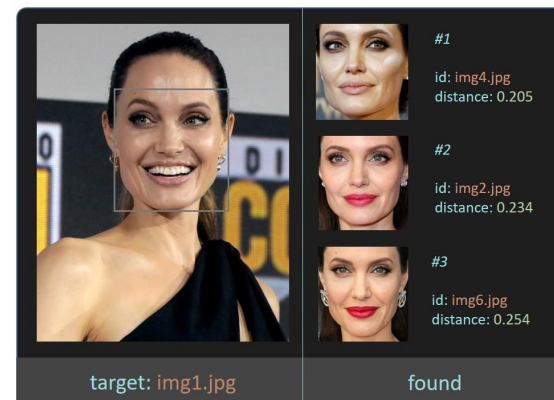


Probe e Gallery

Galleria (Gallery)

si riferisce al database o alla collezione di campioni biometrici memorizzati con cui viene confrontata la sonda. Questi campioni sono solitamente stati pre-elaborati e sono ben documentati, spesso con metadati che possono assistere nel processo di corrispondenza.

Nel contesto del riconoscimento facciale, la galleria sarebbe composta da un insieme di immagini del volto che sono state precedentemente raccolte e possibilmente annotate con informazioni aggiuntive come nomi, ID o altri dettagli identificativi.



Non è perfetto!

- Il fatto che sia molto avanzato o provenga da un ottimo laboratorio non e' detto che sia allora lo strumento migliore.
- Usandolo su esempi pratici ci si rende conto dei limiti
- E' anche un strumento rilasciato gratuitamente e probabilmente sono disponibili modelli molto più avanzati ma che non vengono rilasciati
- Nell'insegnamento invece vedremo come customizzare e rendere più efficaci i vari modelli per adattarli e migliorarli in base alle proprie esigenze applicative

Laboratorio DeepFace: Face analysis

CODICE NON IN ESAME

Studiare invece le osservazioni e nozioni teoriche



Approccio corretto all'uso dei sistemi deep (recall)

- Non è quasi mai conveniente chiedere ad una rete deep di lavorare una grande immagine in ingresso

- Prendi l'immagine e nascondi con un blur i volti dei bambini sotto i 13 anni
- → in un solo passaggio è anche oggi impossibile



- Approccio corretto:
DIVIDI ET IMPERA

- 1) immagine iniziale → Face Detector → Bounding box dei volti
- 2) Crop Volto → Age estimator → Età stimata
- 3) Normale Sfuocatura (filtro passa basso/mediano)

Nota sulle versioni

- Controlla se esistono versioni aggiornate delle librerie, e controlla di installare la versione di tensorflow corretta.

Componente	Versione consigliata
Python	3.10 oppure 3.11
DeepFace	0.0.98
TensorFlow	2.16.x (o superiore compatibile)
NumPy	< 2.0.0 (es. 1.26.4)

- Per esempio (inizio 2026) per un PC locale

```
# 1. Installa TensorFlow e il bridge per il vecchio Keras  
pip install tensorflow>=2.16.1 tf-keras
```

```
# 2. Installa DeepFace (la versione nuova 0.0.98)  
pip install deepface==0.0.98
```

```
# 3. Fondamentale: assicurati che NumPy non sia la versione 2.0  
pip install "numpy<2.0"
```

Installazione e uso (versione Colab)

```
!pip install deepface  
from deepface import DeepFace
```

```
# Carica l'immagine da analizzare  
img_path = "Mark1.jpg"
```

```
# Esegui l'analisi e ottieni i risultati  
result = DeepFace.analyze(img_path,  
actions = ['age', 'gender'])
```

```
# Stampa i risultati  
result
```

```
[{'age': 28,  
  'region': {'x': 68, 'y': 79, 'w': 214, 'h': 214},  
  'gender': {'Woman': 0.10751247173175216, 'Man': 99.89247918128967},  
  'dominant_gender': 'Man'}]
```

```
# Install required packages
!pip install deepface
!pip install opencv-python

# Import Libraries
import cv2
from google.colab.patches import cv2_imshow
from deepface import DeepFace

# Load Haar Cascade Classifier
face_cascade =
cv2.CascadeClassifier(cv2.data.harcascades +
'haarcascade_frontalface_default.xml')

# Read Image
img = cv2.imread("Mark2.jpg")
# img = cv2.imread("face3.jpg")

# Check if image is loaded
if img is None:
    print("Could not open or find the image.")
else:
```

Attenzione di verificare gli output!

Ripartiamo dal nostro
detector di Haar

Attenzione di verificare gli output!

Ripartiamo dal nostro detector

```
# Perform Face Detection
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
faces = face_cascade.detectMultiScale(gray, 1.1, 4)

# Process Each Detected Face
# Process Each Detected Face
for i, (x, y, w, h) in enumerate(faces):
    cv2.rectangle(img, (x, y), (x+w, y+h), (255, 255, 0), 2)
    cv2_imshow(img)

    # Crop the face from the image
    face_crop = img[y:y+h, x:x+w]
    cv2_imshow(face_crop)

    # Use DeepFace to predict age and gender
    predictions = DeepFace.analyze(face_crop, actions = ['age', 'gender'])

    # Now proceed if predictions is a list
    if isinstance(predictions, list):
        # Assuming that each face's prediction is at the same index as the face
        age = predictions[i]["age"]
        gender = predictions[i]["dominant_gender"]

        # Draw the age and gender on the image
        cv2.putText(img, f"Age: {age}", (x, y-10), cv2.FONT_HERSHEY_SIMPLEX, 0.8, (255, 255, 0), 2)
        cv2.putText(img, f"Gender: {gender}", (x, y+h+20), cv2.FONT_HERSHEY_SIMPLEX, 0.8, (255, 255, 0), 2)

# Display the Image
cv2_imshow(img)
```

```

# Install required packages
!pip install deepface
!pip install opencv-python

# Import Libraries
import cv2
from google.colab.patches import cv2_imshow
from deepface import DeepFace

# Load Haar Cascade Classifier
face_cascade = cv2.CascadeClassifier(cv2.data.harcascades + 'haarcascade_frontalface_default.xml')

# Read Image
img = cv2.imread("Mark2.jpg")
# img = cv2.imread("face3.jpg")

# Check if image is loaded
if img is None:
    print("Could not open or find the image.")
else:
    # Perform Face Detection
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    faces = face_cascade.detectMultiScale(gray, 1.1, 4)

    # Process Each Detected Face
    # Process Each Detected Face
    for i, (x, y, w, h) in enumerate(faces):
        cv2.rectangle(img, (x, y), (x+w, y+h), (255, 255, 0), 2)
        cv2_imshow(img)

        # Crop the face from the image
        face_crop = img[y:y+h, x:x+w]
        cv2_imshow(face_crop)

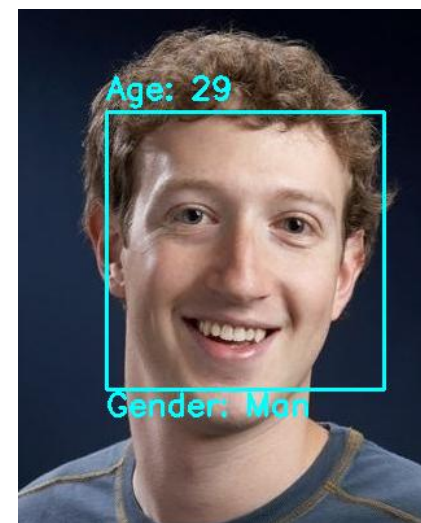
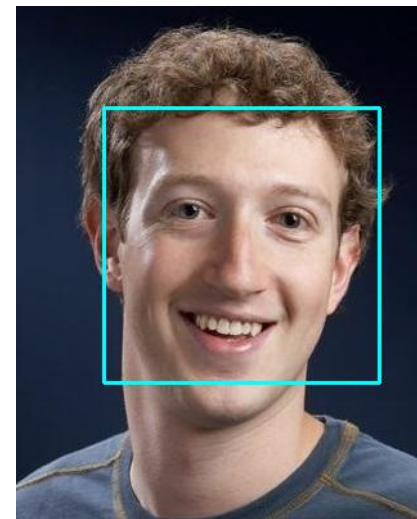
        # Use DeepFace to predict age and gender
        predictions = DeepFace.analyze(face_crop, actions = ['age', 'gender'])

        # Now proceed if predictions is a list
        if isinstance(predictions, list):
            # Assuming that each face's prediction is at the same index as the face
            age = predictions[i]["age"]
            gender = predictions[i]["dominant_gender"]

            # Draw the age and gender on the image
            cv2.putText(img, f"Age: {age}", (x, y-10), cv2.FONT_HERSHEY_SIMPLEX, 0.8, (255, 255, 0), 2)
            cv2.putText(img, f"Gender: {gender}", (x, y+h+20), cv2.FONT_HERSHEY_SIMPLEX, 0.8, (255, 255, 0), 2)

# Display the Image
cv2_imshow(img)

```



```

# Install required packages
!pip install deepface
!pip install opencv-python

# Import Libraries
import cv2
from google.colab.patches import cv2_imshow
from deepface import DeepFace

# Load Haar Cascade Classifier
face_cascade = cv2.CascadeClassifier(cv2.data.harcascades + 'haarcascade_frontalface_default.xml')

# Read Image
img = cv2.imread("Mark2.jpg")
# img = cv2.imread("face3.jpg")

# Check if image is loaded
if img is None:
    print("Could not open or find the image.")
else:
    # Perform Face Detection
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    faces = face_cascade.detectMultiScale(gray, 1.1, 4)

    # Process Each Detected Face
    # Process Each Detected Face
    for i, (x, y, w, h) in enumerate(faces):
        cv2.rectangle(img, (x, y), (x+w, y+h), (255, 255, 0), 2)
        cv2_imshow(img)

    # Crop the face from the image
    face_crop = img[y:y+h, x:x+w]
    cv2_imshow(face_crop)

    # Use DeepFace to predict age and gender
    predictions = DeepFace.analyze(face_crop, actions = ['age', 'gender'])

    # Now proceed if predictions is a list
    if isinstance(predictions, list):
        # Assuming that each face's prediction is at the same index as the face
        age = predictions[i]["age"]
        gender = predictions[i]["dominant_gender"]

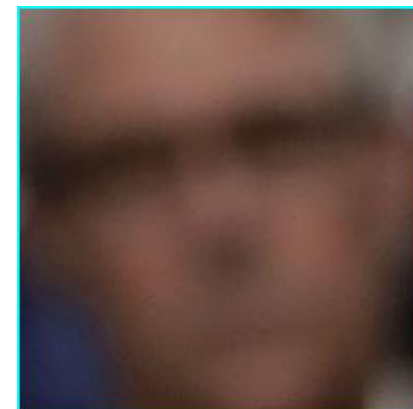
    # Draw the age and gender on the image
    cv2.putText(img, f"Age: {age}", (x, y-10), cv2.FONT_HERSHEY_SIMPLEX, 0.8, (255, 255, 0), 2)
    cv2.putText(img, f"Gender: {gender}", (x, y+h+20), cv2.FONT_HERSHEY_SIMPLEX, 0.8, (255, 255, 0), 2)

# Display the Image
cv2_imshow(img)

```

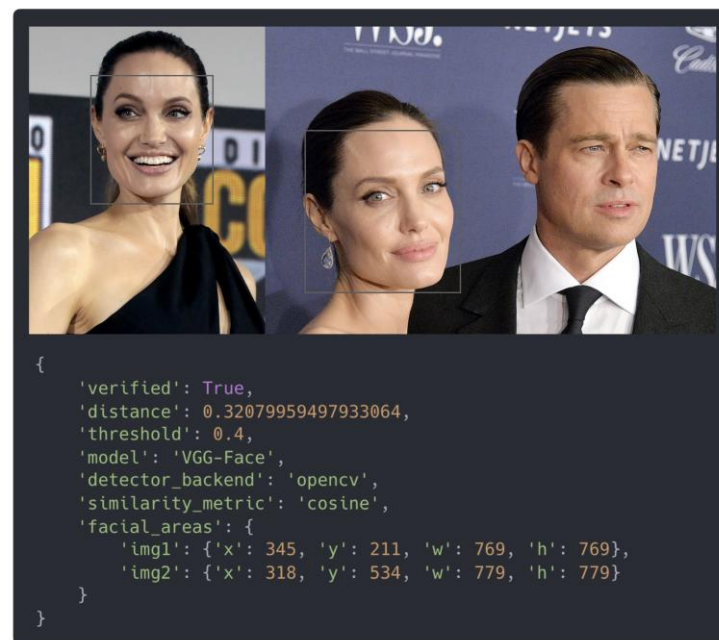
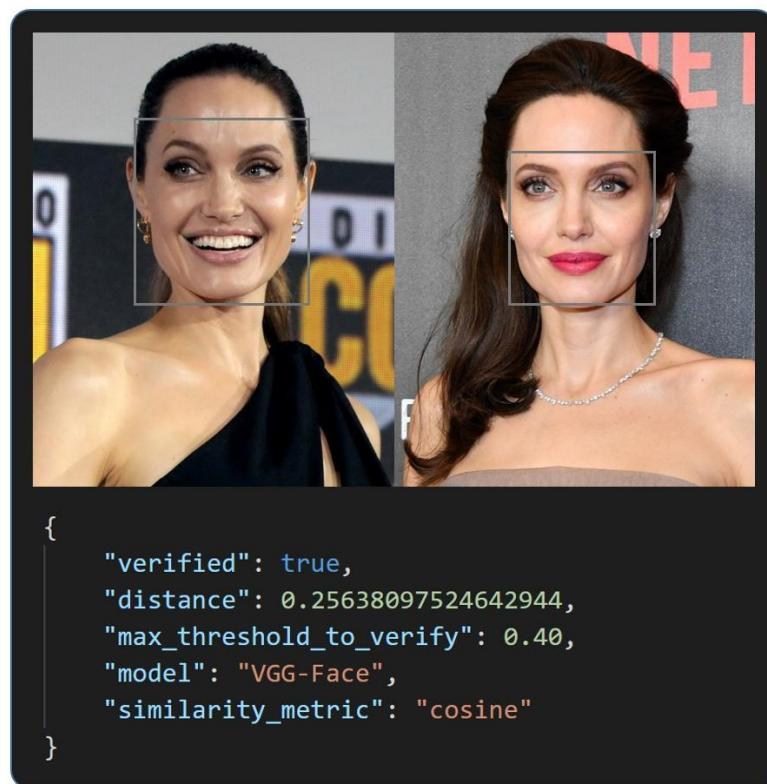
Non sempre le cose vanno come ti aspetti....

- 1) Verificare gli output
- 2) Inserire controlli incrociati....



Verifica della identità

```
result = DeepFace.verify(img1_path = "img1.jpg",  
                          img2_path = "img2.jpg")
```



La funzione di verifica può anche gestire molteplici volti nelle coppie di volti. In questo caso, verranno confrontati i volti più simili tra loro.

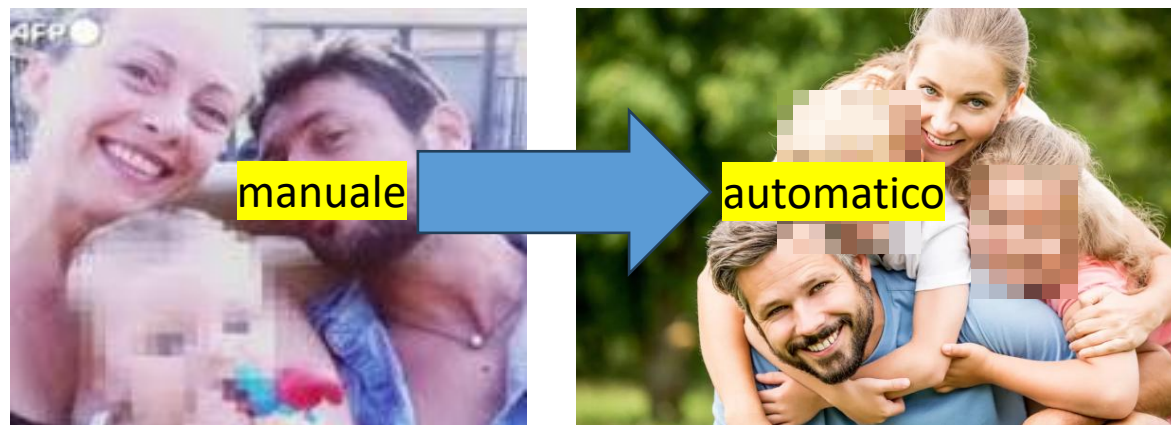
Compito



- Non è un progetto di esame
- Non da punti per l'esame
- Inviare per email al docente 1) codice, 2) immagini di ingresso 3) immagini di uscita prodotte e 4) un pdf di 1 pagina che spiega ingressi, uscite e risultati ottenuti
 - NON ESEGUO CODICI PER VEDERE GLI OUTPUT
- Ma allora perché farli?
 - Per imparare e diventare più operativi
 - In caso di richiesta tesi, aver consegnato dei compiti ben eseguiti, sarà valutato (ma non è garantito)
 - Saltare i punti 3) e 4) non è un buon segnale... non è stato letto il testo con attenzione della consegna...

Compito «BLUR MINORI»

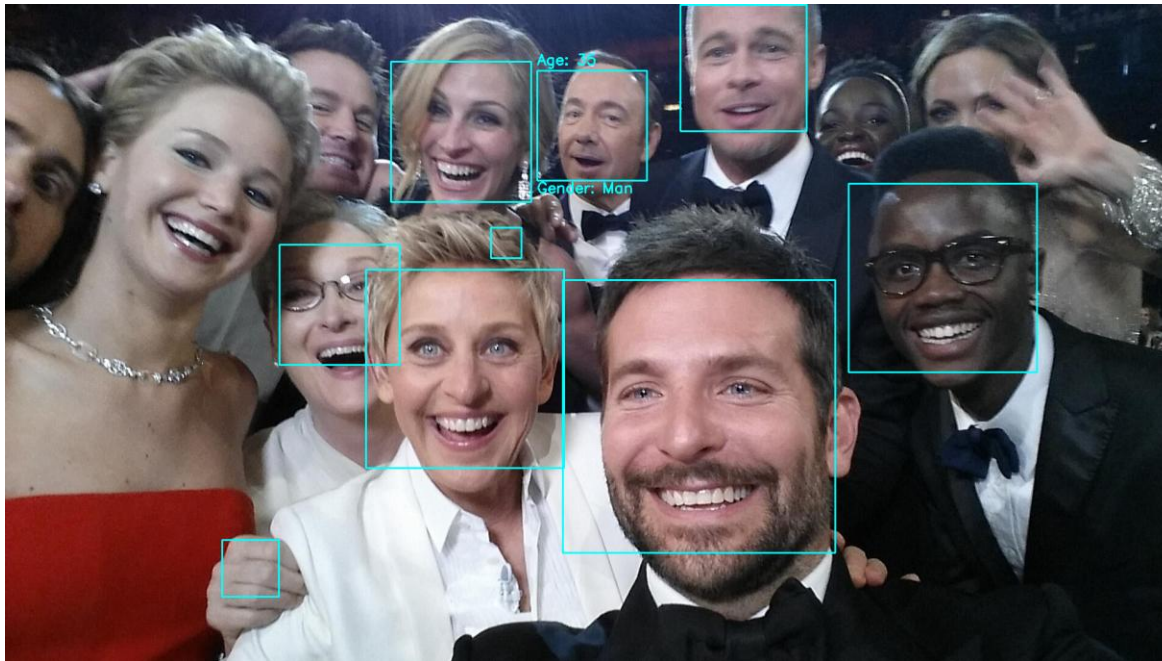
- Unendo le tecnologie già viste, creare uno script Python che
 - legge un'immagine in ingresso
 - trova i volti
 - valuta l'età
 - esegue una sfuocatura dei volto per i minori di 14 anni



Compito

«AGE DETECTOR IN THE WILD»

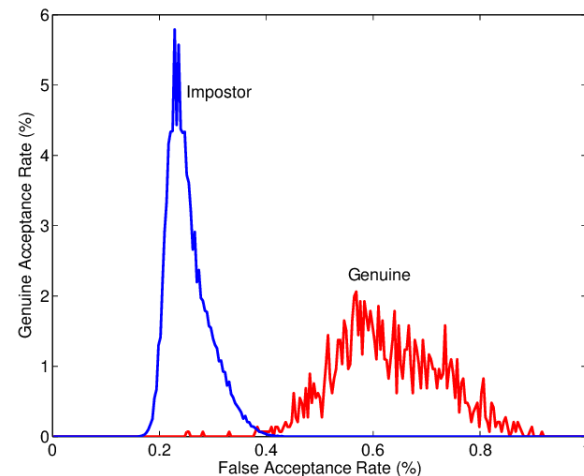
- Raffinare lo script per cercare di analizzare più volti possibili nel file face3.jpg e in altri scenari meno controllati e in the wild



L9

Punti principali visti

- Reti Deep per il volto e Modelli e metodi statistici per la biometria
- Libreria deepFace
- Metodi statistici per la biometria
 - Errore P
 - Distribuzioni
 - DET, ROC
 - FMR, FNRM
 - CMC



Appendici con il codice

Non richiesto all'esame

```

# Install required packages
!pip install deepface
!pip install opencv-python

# Import Libraries
import cv2
from google.colab.patches import cv2_imshow
from deepface import DeepFace

# Load Haar Cascade Classifier
face_cascade = cv2.CascadeClassifier(cv2.data.haarcascades + 'haarcascade_frontalface_default.xml')

# Read Image
img = cv2.imread("face3.jpg")

# Check if image is loaded
if img is None:
    print("Could not open or find the image.")
else:
    # Perform Face Detection
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    faces = face_cascade.detectMultiScale(gray, 1.1, 4)

    # Process Each Detected Face
    for i, (x, y, w, h) in enumerate(faces):
        cv2.rectangle(img, (x, y), (x+w, y+h), (255, 255, 0), 2)

        # Crop the face from the image
        face_crop = img[y:y+h, x:x+w]

        # Skip if face_crop is empty
        if face_crop.size == 0:
            continue

        try:
            # Use DeepFace to predict age and gender
            predictions = DeepFace.analyze(face_crop, actions = ['age', 'gender'])

            if isinstance(predictions, list) and i < len(predictions):
                age = predictions[i]["age"]
                gender = predictions[i]["dominant_gender"]

                # Draw age and gender on the image
                cv2.putText(img, f"Age: {age}", (x, y-10), cv2.FONT_HERSHEY_SIMPLEX, 0.8, (255, 255, 0), 2)
                cv2.putText(img, f"Gender: {gender}", (x, y+h+20), cv2.FONT_HERSHEY_SIMPLEX, 0.8, (255, 255, 0), 2)
            else:
                print(f"Skipping face at {(x, y, w, h)} due to mismatch between detected and analyzed faces.")
        except (ValueError, IndexError) as e:
            print(f"Skipping face at {(x, y, w, h)} due to: {e}")

# Display the Image
cv2_imshow(img)

```

Traccia di codice
da ottimizzare e migliorare!
(vedi compiti)